



Improving network dynamic pricing policies through offline reinforcement learning

Philipp Hausenblas¹ · Dominik Eichhorn¹ · Andreas Brieden¹ · Matthias Soppert² · Claudius Steinhardt²

Received: 21 February 2024 / Accepted: 11 May 2025 / Published online: 19 June 2025
© The Author(s) 2025

Abstract

Due to exponentially growing state and action spaces, network dynamic pricing problems are analytically intractable such that state-of-the-art approaches rely on heuristics. Reinforcement learning has successfully been applied in various complex domains, but its successful applicability to pricing may be limited by two factors. First, the need for extensive state and action space exploration causes lost revenues when directly training within the real world. Secondly, alternatively replicating the real world in an accurate simulation to perform the training therein comes with limitations as well, because calibrating the simulation would require precise domain knowledge, which in general does not exist. To overcome the above issues, with this work, we propose a new dynamic pricing approach based on offline reinforcement learning. In contrast to online reinforcement learning, training solely requires a static data set containing information on historic sales, which stems from applying some arbitrary behavior policy in the past. In particular, we develop a low-dimensional state and actions space reformulation of the considered generic dynamic pricing problem which allows to incorporate the critic-regularized regression algorithm within a scalable approach. We also adapt the standard algorithm's actor loss function, such that it can deal with the pricing problem's state-dependent action space. Our studies show that the trained policy dominates and in some cases substantially outperforms the respective behavior policy. Hence, although there are some limitations that have to be discussed, offline reinforcement learning seems to be a promising approach for dynamic pricing in case online reinforcement learning is not an option.

Keywords Network revenue management · Dynamic pricing · Offline reinforcement learning · Critic-regularized regression

Extended author information available on the last page of the article

1 Introduction

Dynamic pricing refers to a general class of decision making approaches in which a sales process is controlled through adjusting prices over time with the objective to maximize revenue or profit (Elmaghraby and Keskinocak 2003). Even though the distinction is not always sharp, dynamic pricing is part of price-based revenue management (RM) as opposed to quantity-based RM, which comprises capacity allocation decisions (Talluri and van Ryzin 2004a, Chapter 1.1.1). Today, RM is considered as one of the most successful fields of operations research (Talluri and van Ryzin 2004a, Preface) and dynamic pricing plays an important role across various fields – from traditional applications in the airline and hotel industry, via retail and advertisement, to most recent adaptations for cloud computing, e-commerce (Klein et al. 2020a), as well as many others (Özer and Phillips 2012).

An important class of problems considers dynamic pricing over a fixed selling horizon and with limited capacity of unreplaceable resources (Gallego and Topaloglu 2019, Chapter 9). The selling horizon is typically discretized into multiple time periods and in each period, the provider can adjust the price(s) to react to the most current realized sales and to an updated demand forecast. The scientific literature distinguishes between two structurally different types of dynamic pricing problems. In *single-leg* problems (e.g. Bitran and Mondschein 1997), each product consumes exactly one resource and no two products consume the same one. Its generalization, which is more relevant for practice, but at the same time substantially more complex to solve, is the *network* problem. In this problem, products may require multiple resources and different products may access the same resources (e.g. Gallego and van Ryzin 1997). Examples for network problems are airlines selling different trips which share the same flight legs or hotels selling different stays which require the same room type on a particular date. In this work, we focus on a generic formulation of the network dynamic pricing problem which also considers customer choice behavior, i.e., there are multiple products for the same trip (stay) between which customers choose. This problem is characterized by a tremendous computational complexity. As a consequence, it cannot be solved analytically to optimality such that state-of-the-art solution approaches rely on heuristics, e.g., based on solving simplified deterministic models (Pumpensanti and Wang 2020), as suggested for the first time in the seminal papers by Glover et al. (1982) and Dror et al. (1988).

From a methodological perspective, network dynamic pricing problems are stochastic dynamic problems which can be modeled as a Markov decision process. For these types of problems, reinforcement learning (RL) has proven to be a powerful concept to derive effective policies – with spectacular successes in robotic control (e.g. Levine et al. 2016) and complex computer games such as Go (Silver et al. 2016). Following the standard textbook for RL (Sutton and Barto 2018, Chapter 1.1]), the core idea of RL is that an *agent* interacts with an *environment* and thereby gains experience regarding *rewards* that result from taking a particular *action* in a particular *state*. This general concept is schematically depicted in Fig. 1a, applied to a stylized dynamic pricing problem. In this

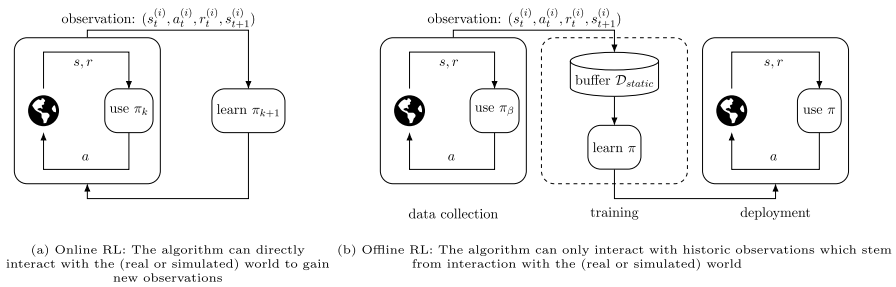


Fig. 1 Difference between online and offline reinforcement learning. Figure adapted from Levine et al. (2020)

context, the environment (globe) is the marketplace, π denotes a pricing policy, a denotes prices originating from that policy, s denotes the state – consisting of remaining time and capacity – and r denotes the realized revenue.

Considering the successes of RL, an obvious idea is to apply RL also to dynamic pricing. Several recent works in the scientific literature pursue this path (e.g. Rana and Oliveira 2014; Bondoux et al. 2020; Kastius and Schlosser 2022). However, we argue that directly applying classical online RL to dynamic pricing comes with limitations that make its applicability to practice difficult. There are two main reasons for this. First, in order to learn an effective policy, RL algorithms require extensive exploration as part of a lengthy training process. Exploration means that vast parts of the state and action space need to be visited in order to determine which states and actions are favorable and which are not. Applied to dynamic pricing, this would mean that the agent would first need to learn that certain prices are unfavorable – causing lost revenues. Clearly, a responsible practitioner cannot accept such loss during the training process in return for a hopefully better policy. In fact, several related works and some of the above named authors acknowledge this issue themselves (e.g., Ferreira et al. 2018; Kastius and Schlosser 2022). Second, to circumvent that, a presumable solution is to model reality in a simulation and then apply traditional RL techniques. While it is true that this avoids lost revenues during training, parameterizing a simulation would require accurate knowledge of reality, e.g., to model customer arrivals or to model the dependency of customers' purchase decisions on the prices which are set. Clearly, researchers and practitioners have put great effort in accurately modeling reality, but the fundamental issue that all resulting policies are subject to the taken modeling assumptions persists. Issues related to model misspecification when applying RL to dynamic pricing have also been discussed several times (e.g. Bondoux et al. 2020). Thus, classical online RL methodologies may face severe limitations regarding their applicability to dynamic pricing.

As for the considered dynamic pricing problem, the issues related to applying classical RL in safety-critical environments or when exploration is costly (healthcare, autonomous driving, etc.) are well-known in literature (Levine et al. 2020). To remedy them, the concept of *offline reinforcement learning* has been developed and it has gained traction in recent years (e.g. Kumar et al. 2019; Kumar et al. 2020; Monier et al. 2020). As visualized in Fig. 1b, the core difference between classical

online RL and offline RL is that in offline RL, the agent has no means to interact with the environment to learn a policy. Instead, the only source for learning a policy π is a static data set which was generated with some (potentially unknown) behavior policy π_β in the past. Intuitively speaking, an offline RL algorithm attempts to extract, recombine and generalize good components of this behavior policy. Generalization in this context means inferring that an action that yields high rewards in one situation might also yield high rewards in other, similar situations. Thus, if successful, offline RL provides a means to improve a currently best known behavior policy while entirely avoiding the risk of a potentially costly online training due to lost revenues. Importantly, most of the offline RL algorithms do not require information about the environment in form of a model. That is, in the context of dynamic pricing, neither information about customer arrival probabilities nor about the customer choice behavior is necessary. It has to be noted, however, that the success of offline RL depends on the static data set, in particular on its coverage on high-reward region which the algorithm can learn from.

In conclusion, as opposed to online RL, offline RL can in principle be applied to dynamic pricing. However, up to now, the scientific literature – to the best of our knowledge – has not answered the research question how this can be achieved. With this work, we contribute to filling this gap. In particular, the contributions of this work are the following:

- We answer the research question by demonstrating how the considered network dynamic pricing problem can be reformulated with a low-dimensional state- and action-space representation, to allow for the application of so-called actor-critic RL (online and offline) algorithms – like critic-regularized regression which we use. Only due to this reformulation, these algorithms scale to real-life problem instances. In addition, we propose an adaptation of the standard algorithm's actor loss function. This is necessary to handle the particularity that the considered dynamic pricing problem has a state-dependent action space, because certain products' resources might be depleted and cannot be offered.
- Compared to the dynamic pricing problems addressed in existing works that propose online RL-based methods, the network dynamic pricing problem that we consider (multi-products, perishable multi-resources, discrete prices, finite selling horizon, customer choice) is much more generic and complex. Further, in terms of the pricing problem, our work directly addresses what recent closely related works on RL-based dynamic pricing and capacity control state for future research (Bondoux et al. 2020; Kastius and Schlosser 2022).
- Based on a systematic numerical study, we discuss characteristics of behavior policies that generate the sales data that are beneficial for applying offline RL. The knowledge of these characteristics allows practitioners to assess the potential of applying offline RL to the data generated with their current policy. It further allows to conclude how their current policy should be adapted to obtain sales data that is a more promising basis for applying offline RL.
- Despite the large potential, we discuss prospective limitations of offline RL and the specific critic regularized-regression approach in the context of dynamic pricing. This allows practitioners and researchers building on this work to avoid

potential pitfalls and to address relevant questions in future research, respectively.

- Most importantly, we demonstrate that offline RL can be successfully applied to dynamic pricing. This is done by generating sales data with different behavior policies in a simulation, by training the critic-regularized regression offline RL algorithm on this data, and by evaluating the trained policy in the simulation against the behavior policy.

The remainder of this paper is organized as follows. In Sect. 2, we provide the relevant methodological background on Markov decision processes as well as on classical (online) RL and offline RL. It also contains a high-level explanation of the critic-regularized regression algorithm. This background knowledge is required to be able to follow the subsequent sections. In Sect. 3, we review the related literature on RL-based dynamic pricing and capacity control. The considered network dynamic pricing problem is presented in Sect. 4, followed by the presentation of the behavior policies used to generate the sales data and the developed offline RL policy in Sect. 5. Sect. 6 contains the numerical study and the derivation of managerial insights. In Sect. 7, we conclude the work and give an outlook on future research.

The reader familiar with the methodological backgrounds in Sect. 2 may skip the respective subsections.

2 Methodological background

The purpose of this section is to present the basic concepts of offline RL, including an intuitive description of critic-regularized regression, and to explain the differences to online RL. This requires an understanding of online RL. For this purpose, we first briefly describe Markov decision processes (Sect. 2.1), which lay the foundation for the mathematical formulation of RL. Next, we state the core ideas of online RL and its major types of algorithms (Sect. 2.2) on which offline RL is built upon, including actor-critic methods that form the basis for critic-regularized regression. The section on offline RL (Sect. 2.3) contains a discussion of its core concepts, algorithm types, main ideas of the specific critic-regularized regression algorithm, as well as methods for performance evaluation.

Readers familiar with Markov decision processes as well as classical (online) RL may skip these parts.

2.1 Markov decision process

The mathematical formulation of optimization problems tackled with RL is based on the concept of a Markov decision process (MDP) (Sutton and Barto 2018, Chapter 3). In an MDP, at each time step t , the *agent* observes a state $s_t \in \mathcal{S}$ of the *environment*, where $\mathcal{S}$ is the set of possible states that the environment can be in. Based on this information, the agent takes an action $a_t \in \mathcal{A}$, where $\mathcal{A}$ is the set of available actions from which the agent can choose. Then, the environment reacts

by returning a reward of $r_t \in \mathbb{R}$ to the agent and the environment transitions to a new state $s_{t+1} \in \mathcal{S}$. Formally, an MDP is defined as a tuple $(\mathcal{S}, \mathcal{A}, p, d_0, \gamma)$, where $p : \mathcal{S} \times \mathbb{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$ is a conditional probability distribution of the form $p(s', r|s, a) = P(s_{t+1} = s', r_t = r | s_t = s, a_t = a)$. Thus, it denotes the probability of observing a reward of r and transitioning to state s' at the beginning of time step $t + 1$ when picking action a in state s at time step t . As required by the Markov property, p fully formalizes the one-step-transition dynamics including the reward mechanism of the MDP in terms of probabilities. Further, d_0 is the probability distribution of the initial state s_0 and $\gamma \in (0, 1]$ is a factor for discounting future rewards.

A policy $\pi : \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is a conditional probability distribution $\pi(a|s)$ which specifies the probability that the agent, when being in state s , will take action a . A trajectory τ of length $H \in \mathbb{N}$ is an alternating sequence consisting of $H + 1$ states, actions and rewards, respectively, given by $\tau = (s_0, a_0, r_0, \dots, s_H, a_H, r_H)$. Thus, for a given MDP with fixed initial state distribution and transition dynamics, the trajectory distribution dependent on a policy π is defined as $p_\pi(\tau) = d_0(s_0) \prod_{t=0}^H \pi(a_t|s_t) p(s_{t+1}, r_t | s_t, a_t)$. This trajectory distribution $p_\pi(\tau)$ describes the probability of observing a trajectory τ when the agent picks actions according to policy π .

An important concept within the framework of MDPs is that of value functions. For a given policy π , the state-(action-)value function $V^\pi(s)$ ($Q^\pi(s, a)$) is the expected sum of discounted rewards when starting in state s (taking action a) and subsequently following policy π . The state-action-value function is often referred to as Q-value function.

Using the above notations, the problem's objective tackled with RL can formally be written as

$$\pi^* = \arg \max_{\pi} J(\pi) = \arg \max_{\pi} \mathbb{E}_{\tau \sim p_{\pi}} \left[\sum_{t=0}^H \gamma^t r_t \right] = \arg \max_{\pi} \mathbb{E}_{s_0 \sim d_0, a_0 \sim \pi(\cdot | s_0)} [Q^\pi(s_0, a_0)].$$

That is, the goal is to find a policy π^* which maximizes the objective function $J(\pi)$. $J(\pi)$ denotes the expected value of the sum of discounted rewards, where the rewards come from trajectories that are sampled from the trajectory distribution induced by policy π .

To solve this stochastic dynamic optimization problem exactly, assuming that the one-step-transition dynamics p of the MDP are known, one may resort to dynamic programming. In most applications, however, the dynamics of the MDP are unknown or state space and/or action space are too large. In this case, RL algorithms may be used.

2.2 Online reinforcement learning

As briefly described in Sect. 1, the core idea of RL (Sutton and Barto 2018) is that an agent interacts with an environment, thereby gaining experiences regarding rewards resulting from following a particular policy. This experience allows to learn a better policy with higher expected rewards. More specifically, most of the online

RL algorithms proceed roughly according to the following scheme known as generalized policy iteration (Sutton and Barto 2018, Chapter 4.6):

- *Step 1:* The current policy π_0 is randomly initialized.
 - *Step 2:* Multiple sample trajectories are generated by the agent interacting with the environment following the current policy π_k , resulting in a total of $n \in \mathbb{N}$ transitions. That is, the state of the environment s_t is observed and an action a_t is chosen according to π_k . This results in a reward of r_t and in the transition to a new state s_{t+1} . Interaction and observation are repeated n times such that the procedure generates a data set of *in total*, meaning across all trajectories, n transitions $\mathcal{D} = \{(s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, s_{t+1}^{(i)})_{i=1, \dots, n}\}$. This is schematically depicted in the “inner loop” in Fig. 1a. To learn an effective policy, it is important that the interaction with the environment ensures a certain level of exploration. Exploration refers to trying out actions that are omitted by the current policy because it does not consider them to be maximizing the objective function. However, these actions might turn out to result in higher rewards than the currently best known action – even if this comes with the risk of temporarily reducing rewards. Without exploration, it might be impossible to discover the optimal policy.
 - *Step 3:* The generated data is used to determine an improved policy π_{k+1} which is then set as the new current policy. The nature of this update depends on the specific RL algorithm and it is schematically depicted in the “outer loop” in Fig. 1a. The algorithm loops over Step 2 and Step 3. There are different types of online RL algorithms that differ in various regards. In order to outline actor-critic methods, which form the basis for critic regularized regression, we first describe in the following the core concepts for updating the policy (Step 3) of value-based methods, temporal difference learning methods, and policy gradient methods:
- *Value-based methods:* Value-based methods (Sutton and Barto 2018, Part I) estimate value functions, mostly Q-value functions (see Sect. 2.1), to implicitly derive a policy. For example, given a Q-value function estimate, one can extract a policy by choosing, in each state, the action which is associated with the largest estimated Q-value with a probability of $1 - \epsilon$ and some other random action with probability of ϵ to ensure some exploration. SARSA (Rummery et al. 1994) is an example of such a value-based method.
 - *Temporal difference learning methods:* Temporal difference learning methods (Sutton and Barto 2018, Chapter 6) are specific kinds of value-based methods, in which value function estimates are updated iteratively using the Bellman-operation, also referred to as bootstrapping. For instance, in Q-learning this operation for a given transition i takes the form

$$Q^\pi(s_t^{(i)}, a_t^{(i)}) \leftarrow (1 - \alpha)Q^\pi(s_t^{(i)}, a_t^{(i)}) + \alpha(r_t^{(i)} + \gamma \max_{a \in \mathcal{A}} Q^\pi(s_{t+1}^{(i)}, a)),$$

where α denotes the learning rate. The updated estimated Q-value is a weighted mean (α) of the current estimated Q-value and the one-step bootstrap estimate of the Q-value. The one-step bootstrap estimate of the Q-value for state s_t and

action a_t is intuitively obtained as follows. First, the reward is observed which is received when taking action a_t in state s_t . When arrived in the next state s_{t+1} , the action with the highest estimated Q-value is selected. Thus, its (discounted) current estimated Q-value is added to the previously received reward. For large and, in particular, continuous state spaces, the Q-value function needs to be parametrized. This is because, on the one hand, the above update has to be performed for every state-action pair, which becomes computationally intractable if the state spaces grows large. On the other hand, even if this was no limitation, the sparse observations of specific states lead to an enormous amount of time and data required to obtain accurate Q-value function estimates for every state-action pair. Parameterization enables generalization, i.e., similar states can be used to predict Q-values of a state unobserved to this point. In most cases, this is done by using a neural network. Deep Q-Network (DQN) (Mnih et al. 2015) is an example of such a temporal difference learning method.

- *Policy gradient methods:* Policy gradient methods (Sutton and Barto 2018, Chapter 13), in contrast to value-based methods, explicitly model parametric policies π_θ . For instance, a parametric policy might be represented by a neural network which takes the state s as input and returns a probability distribution over the set of available actions. In this case, θ denotes the parameter vector specifying all of the network's weights and biases. In order to obtain an improved policy, θ is adjusted such that the resulting policy puts more probability mass on actions that have a high Q-value, i.e., are assumed to result in high rewards. This is done by estimating the gradient of the objective function J with respect to θ which also involves estimating the Q-values, using the samples in the data set $\mathcal{D}$. In standard policy gradient such as REINFORCE (Williams 1992), the Q-values are approximated using Monte-Carlo estimates. Subsequently, a gradient ascent step is taken in order to update the policy.
- *Actor-critic methods:* Actor-critic methods (Sutton and Barto 2018, Chapter 13.5) extend the idea of policy gradient methods by using temporal difference learning to estimate the Q-values. As for all policy gradient methods, the policy (denoted as actor) is parameterized, which mostly is done by a neural network. In addition to standard policy gradient methods, the Q-value function (denoted as critic) which evaluates the actor's policy is parameterized, mostly also via a neural network Q_θ with parameter vector θ , and estimated using temporal difference learning. The learning of critic and actor happens in parallel. Advantage Actor-Critic (A2C) (Mnih et al. 2016) exemplifies a type of actor-critic method.

2.3 Offline reinforcement learning

As in online RL, the goal in offline RL is to maximize the objective function J . However, in contrast to online RL, offline RL cannot iteratively generate new sample trajectories by applying the current policy in the environment (Step 2 in online RL). Instead, it can only access (sample batches of) a static data set $\mathcal{D}_{static}$ that was generated under an unknown behavior policy π_β , without the possibility of further exploration (Prudencio et al. 2023). Regarding the three steps of online RL algorithms presented in Sect. 2.2, offline RL algorithms can only carry out the policy

initialization (Step 1 in online RL) as well as the policy update (Step 3 in online RL). The policy update is typically executed multiple times, with different batches sampled from the static data set. Thus, offline RL tries to find the best policy π that is supported by the static data set $\mathcal{D}_{static}$. Consequently, if $\mathcal{D}_{static}$ does not contain transitions of high-reward state space regions, it may be impossible to discover them (Levine et al. 2020). The restrictions of offline RL entail two main challenges described in the following (Levine et al. 2020):

- *Distributional shift*: The purpose of offline RL is to learn a policy which outperforms the behavior policy. Thus, by design, the learned policy shall differ from the behavior policy. However, when straightforwardly applying classic online RL algorithms to a static data set, the learned policy might deviate *largely* from the behavior policy. One cause is that the trained value estimators extrapolate from the observed data, might assign high values to out-of-distribution actions (i.e., actions that are not in the static data set), which then are chosen due to their high value estimates. Thus, also the trajectory distributions induced by the two policies might deviate largely. This is known as distributional shift. In extreme cases, applying the learned policy would lead to visiting states which have not been visited when applying the behavior policy. The issue with distributional shift lies in the resulting inaccurate value estimates. That is, for estimating the value function, only samples that were generated under the trajectory distribution induced by the behavior policy can be used. However, the resulting estimated value function might not be accurate when evaluated on samples coming from a trajectory distribution induced by a policy other than the behavior policy. An analogy from supervised learning is that if a model is trained on samples from one distribution (of the independent variable), but evaluated on samples of a different distribution, there are no performance guarantees regarding the model's accuracy. As a consequence, when evaluating a learned policy, the inaccurate value estimates may lead to suboptimal actions. Thus, offline RL algorithms should be designed to strike a balance between deviating from the behavior policy to realize improvements, while at the same time limiting the distributional shift to ensure accurate value function estimates.
- *Value overestimation*: Inaccurate value function estimates, e.g. due to distributional shift, lead to RL algorithms updating the policy towards actions with incorrectly high reward estimates. In the classical online RL setting, meaning with the possibility to interact with the environment, this issue is addressed: If an algorithm overestimates the value of a certain action, it is more likely to choose that action during the next interaction with the environment. It will observe that the reward of that action is not as high as estimated and, thus, correct its estimate downwards. This downward correction is not possible when there is just one static data set available, as in the offline RL setting. Thus, offline RL algorithms should be designed not to assign high values to out-of-distribution actions, because it is not certain that these actions actually yield high rewards.

Apart from being able to overcome these challenges, offline RL algorithms should have the following desirable properties (Prudencio et al. 2023):

- *Filtering*: Offline RL algorithms should not simply imitate the behavior policy, but filter out bad behaviors from the static data set.
- *Generalization*: Offline RL algorithms should recognize that behavior that is good in a certain situation could also be good in other similar situations.
- *Compositionality*: Offline RL algorithms should extract locally good behaviors from several overall suboptimal trajectories and combine them into an overall better policy.

As for online RL algorithms, there are also different types of offline RL algorithms and they are designed to address the two challenges (distributional shift, value overestimation) described above (Monier et al. 2020):

- *Explicit policy constraint methods*: To limit the distributional shift, the learned policy is constrained in its deviation from the behavior policy, measured by some appropriate metric.
- *Q-values regularization methods*: To prevent value overestimation on out-of-distribution actions in methods that make use of Q-value functions, high Q-value estimates for these actions are penalized during the training.
- *Implicit policy constraint methods*: To prevent the learned policy from choosing out-of-distribution actions which may result in bad performance, low reward actions are filtered out of the data set and then the policy is trained to imitate only the high reward actions contained in the data set.

A specific offline RL algorithm which we build upon in our work, and which we describe in detail in Sect. 5.2, is critic-regularized regression (CRR) (Wang et al. 2020). It belongs to the implicit policy constraint methods and has the same basic structure as actor-critic methods (see Sect. 2.2). Intuitively, it conducts a filtering on sampled transitions such that only actions which are likely to be better than the current policy are used to update the actor. Then, the critic update is performed using the current policy.

Regarding performance evaluation for offline RL methods after having learned a policy, there are two fundamentally different approaches (Levine et al. 2020):

- *Online evaluation*: In online evaluation, the agent interacts with the environment, exploiting the learned policy and observing the resulting rewards.
- *Offline evaluation*: The alternative, offline or so called off-policy evaluation methods, allow to assess the performance of the learned policy without any interaction with the environment. In essence, these methods estimate the expected sum of discounted rewards under the trajectory distribution induced by the learned policy while having only samples from the trajectory distribution induced by the behavior policy that are part of the static data set. This can be done with the help of importance sampling or enhancements thereof such as weighted importance sampling (Precup et al. 2000). Furthermore, by using just the static data set it is possible to give confidence bounds on the performance of the learned policy (Thomas et al. 2015).

For further general background on offline RL, we refer to the survey and tutorial papers by Levine et al. (2020), Monier et al. (2020), and Prudencio et al. (2023).

3 Related literature

In this section, we review the closest related literature, i.e., publications that use RL in applications of dynamic pricing and capacity control (also denoted as availability control). Dynamic pricing problems with discrete prices and capacity control problems can be transferred into each other (Gallego and van Ryzin 1997), because setting a certain price can be considered as making available (or accepting requests of) a specific product with a certain price (see Strauss et al. (2018)). We summarize the considered problem as well as the applied RL algorithm in Table 1. Further relevant references with regard to the literature on RM are integrated in the problem statement in Sect. 4, and references regarding the adapted policies in Sect. 5.

The first work to propose RL in a RM problem is Gosavi et al. (2002). The authors consider a single-leg capacity control problem in which incoming customer requests for one of the multiple fare classes are either accepted or rejected. The problem is modeled as a Semi-Markov Decision Process which, in contrast to a standard MDP, allows to model that the system can remain in a certain state for a specific time. The authors present a new RL algorithm, denoted as λ -SMART. Its name stems from the fact that it is a variation of the temporal difference algorithm $TD(\lambda)$ (Sutton and Barto 2018, Chapter 6) as well as from the fact that the Semi-Markov Average Reward Technique is applied, which sums the temporal differences over the entire trajectory. The work reports that the RL algorithm outperforms the widely used nested EMSR (EMSR-a) heuristic (Talluri and van Ryzin 2004a, Chapter 2.2.4).

Based on this seminal work, Gosavi (2004) develops another RL algorithm for the same single-leg capacity control problem. The new algorithm, denoted as Q-P-learning, is an adaptation of standard Q-learning (Sutton and Barto 2018, Chapter 6.5) to allow for policy iteration in Semi-Markov Decision Processes with average rewards. The work shows that Q-P-learning outperforms Q-learning and the above-named EMSR heuristic.

Several years later, Rana and Oliveira (2014) consider a single-leg dynamic pricing problem with one product and discrete prices. The authors state that the decision-maker in practice typically does not possess full knowledge of the two main sources of randomness in pricing problems, i.e., the customer arrival rate and the demand function. This motivates the authors' development of an algorithm called $Q(\lambda)$ that combines standard Q-learning with the concept of eligibility traces, in which the weighted rewards of multiple following steps are incorporated in the policy update. Their results show that the proposed algorithm outperforms standard Q-learning.

Lawhead and Gosavi (2019) consider a single-leg capacity control problem in which the decision-maker decides on the fare to open. Arriving customers then either accept or reject this offer. The authors develop a new actor-critic RL algorithm in which the actor values get bounded to prevent computer overflow, which may arise as actor values grow large. As the works above, they demonstrate its dominance with regard to the nested EMSR heuristic.

Among the first authors to acknowledge limitations of applying RL-based algorithms directly to real-world RM problems are Bondoux et al. (2020), as it

Table 1 Closest related works, considering RL-based dynamic pricing and capacity control

Paper	Dynamic pricing or capacity control problem					reinforcement learning		
	Dynamic pricing or capacity control problem					Selling horizon	Online	Offline
	Resource(s)	Inventory	Product(s)	Demand	Pricing/capacity control			
Gosavi et al. (2002)	Single	Perishable	Multiple	Independent	Capacity control	Finite	SMART (with $TD(\lambda)$)	✗
Gosavi (2004)	Single	Perishable	Multiple	Independent	Capacity control	Finite	Q-P-learning	✗
Rana and Oliveira (2014)	Single	Perishable	Single	n.a.	Discrete prices	Finite	Q-learning, $Q(\lambda)$	✗
Lawhead and Gosavi (2019)	Single	Perishable	Multiple	Independent	Capacity control	Finite	(bounded) actor-critic	✗
Bondoux et al. (2020)	Single	Perishable	Single	n.a.	Discrete prices	Finite	Q-learning, DQN	✗
Kastius and Schlosser (2022)	Single	Replenishable	Multiple	Dependent	Discrete (DQN), continuous (SAC)	Infinite	DQN, SAC	✗
this work	Multiple	Perishable	Multiple	Dependent	Discrete prices	Finite	✗	CRR

“requires costly interactions with actual customers” due to the need for exploration. Thus, the authors propose to pre-train an RL-algorithm by interactions with the existing revenue management system (RMS), and to subsequently continue learning in the real world. They consider a single-leg dynamic pricing problem with discrete prices in a monopoly and a duopoly setting. In their studies, the authors evaluate Q-learning and the Deep Q-learning algorithm DQN (Mnih et al. 2015). They show that DQN can be successfully pre-trained with the RMS, reducing the costly training with random initialization.

Quite recently, Kastius and Schlosser (2022) consider single-leg dynamic pricing with discrete prices in duopoly and oligopoly markets, to focus on analyzing the behavior and performance of RL algorithms under competition. The two algorithms applied are Soft Actor-Critic (SAC) (Haarnoja et al. 2019) and the above-named DQN. Both algorithms learn effective pricing strategies in the complex scenarios tested, but the authors also note both algorithms’ limitations in terms of applicability, due to the enormous required training.

Finally, there are a few less related works in which either the pricing problem (e.g. no finite selling horizon and unlimited resources) or the learning-based algorithm (e.g. multi-armed bandit approach) differs substantially from the ones considered in this work. We refer to Raju et al. (2006); Ferreira et al. (2018); Misra et al. (2019); Bu et al. (2020), and those not discussed above but summarized in Kastius and Schlosser (2022). We also refer to the survey paper on “dynamic pricing and learning” by den Boer (2015), in which the presented works consider problems and/or methodologies that cover a broad range, e.g., Bayesian approaches to learn parameters of an unknown demand function.

In summary, several works propose and assess RL-algorithms for dynamic pricing or capacity control problems. However, to the best of our knowledge, all existing works in this context consider single-leg problems, presumably because of the limitation that network problems grow too quickly in size for the proposed algorithms. For the single-leg problems, various online RL algorithms are shown to perform well under the given circumstances, also compared to benchmarks from literature. Yet, the most recent works also discuss limitations of applying RL directly to reality, due to costly exploration. In our work, we address these limitations. One the one hand, we demonstrate how the offline RL algorithm critic-regularized regression can successfully be used to train an effective dynamic pricing policy, without the need to interact with reality. On the other hand, we consider a very generic network dynamic pricing problem (multiple resources, multiple products, discrete prices), which requires a problem reformulation that can be used in a similar manner to allow a scalable application of other RL algorithms.

4 Problem statement

In this section, we formalize the addressed problem. We begin with the problem statement and relate it to the problems discussed in other works. We use notation common in RM and, at the end of this section, state how it links to the RL notation introduced in the previous section.

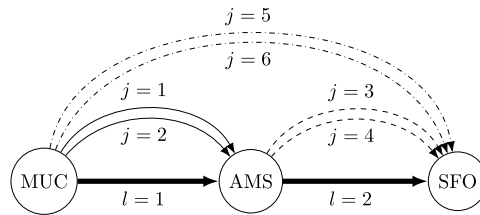


Fig. 2 Transportation network with three nodes, two legs $l \in \{1, 2\}$ (thick arrows), and three markets $k \in \{1, 2, 3\}$ with two products (thin arrows with different line style per market) each, i.e., $n_1 = n_2 = n_3 = 2$, and $\mathcal{J}_1 = \{1, 2\}$, $\mathcal{J}_2 = \{3, 4\}$, $\mathcal{J}_3 = \{5, 6\}$

We consider a generic network dynamic pricing problem with discrete prices and customer choice as it, e.g., arises in transportation networks like in the airline industry. While we frame the following problem statement for such transportation application, we note that its generic formulation allows much wider application, e.g., in the hotel or car rental industry (see Sect. 1).

As depicted in Fig. 2, the network consists of multiple origin-destination pairs, denoted as markets $k \in \mathcal{K} = \{1, 2, \dots, K\}$, for which the provider offers transportation. In a market k , there are n_k different products and the set of products in market k is defined by $\mathcal{J}_k = \{\sum_{k' < k} n_{k'} + 1, \sum_{k' < k} n_{k'} + 2, \dots, \sum_{k' \leq k} n_{k'}\}$. The set of all products is $\mathcal{J} = \bigcup_{k=1}^K \mathcal{J}_k$, and the number of all products is $n_{\mathcal{J}} = \sum_{k=1}^K n_k$. Products of the same market can differ in their itinerary, e.g., with or without stopover (not depicted) and they can differ in their specifics, e.g., with or without free cancellation. For the provider, selling a product requires available resources on one or multiple of the L network's legs $l \in \mathcal{L} = \{1, 2, \dots, L\}$, as defined by the resource consumption matrix $M = (m_{j,l})_{j \in \mathcal{J}, l \in \mathcal{L}}$ which links products to legs. More specifically, the element $m_{j,l} = 1$ if product j consumes a resource unit on leg l , and $m_{j,l} = 0$ if not. M_j denotes the j -th row of the resource consumption matrix.

The considered time frame is discretized into disjunct periods $t \in \mathcal{T} = \{1, 2, \dots, T\}$, as common in the literature (Gallego and Topaloglu 2019, Chapter 9). At the beginning of each period, the provider sets a price, followed by a potential customer arrival and potential purchase decision (see Fig. 3).

For each product j , there is a discrete set of admissible price points (or fare levels), denoted as $\mathcal{F}_j = \{f_{j,1}, f_{j,2}, \dots, f_{j,Z_j}\}$, with the corresponding product-specific price index set $\mathcal{Z}_j = \{1, 2, \dots, Z_j\}$. The provider's decision at the beginning of each period t is to set a price vector x_t which comprises the prices $x_{t,j}$ for all products $j \in \mathcal{J}_k$ of all markets $k \in \mathcal{K}$, i.e., $x_t = (x_{t,j} : j \in \mathcal{J}_k, k \in \mathcal{K})^T$. Thus, the number of possible prices for each market k is $\prod_{j \in \mathcal{J}_k} |\mathcal{F}_j|$ and when assuming $\mathcal{Z}_i = \mathcal{Z}_j \forall i, j \in \mathcal{J}_k$, it is $|\mathcal{F}_j|^{|\mathcal{J}_k|}$. Importantly, after setting the price vector, the provider cannot observe, whether a customer arrives. The provider can only observe, whether, and if so, which product was purchased. In particular, if no product has been purchased, the provider does not know whether no customer has arrived or whether a customer has arrived but decided not to purchase.

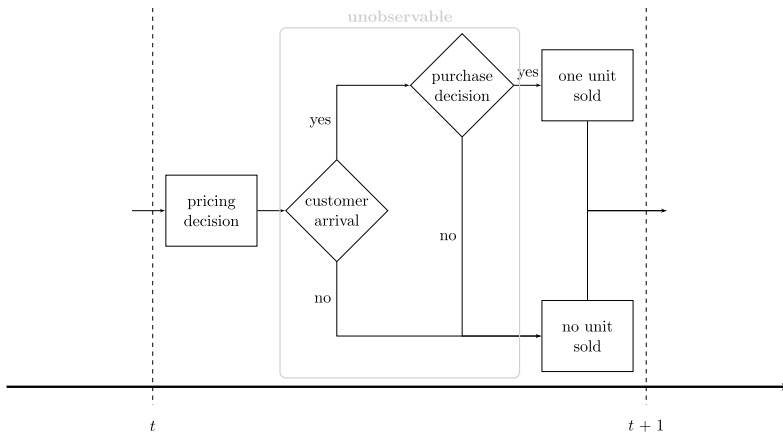


Fig. 3 Schematic depiction of the sales process. Each period consists of potentially three subsequent events, i.e., pricing decision, (potential) customer arrival, and (potential) purchase decision. The (potential) customer arrival and the (potential) purchase decision are not observable to the provider. After the pricing decision, the provider can only observe whether and, if so, which product was purchased in a period

At the beginning of a period t , the remaining capacity on each leg is $c_{t,l}$. We define the period-specific remaining capacity vector as $c_t = (c_{t,1}, c_{t,2}, \dots, c_{t,L})$. The initial capacity on leg l is C_l , i.e., $c_{1,l} = C_l \forall l \in \mathcal{L}$. With probability $\lambda_{t,k}$, a customer who is interested in buying a product of market k arrives in period t . Further, $\lambda_{t,0}$ denotes the probability that no customer arrives, such that $\sum_{k \in \mathcal{K}} \lambda_{t,k} + \lambda_{t,0} = 1 \forall t \in \mathcal{T}$.

Regarding the formalization of the customer-choice behavior, $q_{t,j}(x_t)$ denotes the probability that a customer who arrives in period t purchases product j at the set prices. The specification of $q_{t,j}$ depends on the actual customer-choice behavior model and further considers that certain products might not be available due to depleted resources. Finally, for notational convenience, we define an additional price $f_{j,0}$ (with index $z = 0$) for each product j that denotes a sufficiently high price (reservation price) for which demand is zero, corresponding to not offering the product.

Noting the above-named fact that pricing problems can equivalently be considered as capacity control (“allocation”) problems, and vice versa (Gallego and van Ryzin 1997), the stated problem links to problems addressed in literature as follows. The problem can be considered as a combination of the “pricing/allocation” problem in a “multiple-market, multiple-resource setting” discussed in Gallego and van Ryzin (1997), with customer choice consideration and with discrete prices. It can also be considered as the network generalization of the dynamic single-leg discrete pricing problem with multiple products addressed in Schön (2010) and is listed as such in her Sect. “Extensions”. Further, it is the multiple-product generalization of the dynamic network discrete pricing problem with a single product (per leg) addressed in Erdelyi and Topaloglu (2011) and Barz et al. (2023). In the context of their proposed deterministic linear program (DLP), Erdelyi and Topaloglu (2011) discuss a dependent demand generalization of the single product problem in which buying probabilities for an itinerary also depend on the prices for different itineraries. while this generalization does not exactly resemble

the stated problem above, their respective DLP formulations are structurally identical, as we elaborate in Sect. 5.1.3.

The mapping between the (RL) notation used in Sect. 2 and the (RM) notation introduced in this section is as follows. The state s_t is defined by a combination of period t and capacity vector c_t and the set of all possible combinations of period and capacity defines the problem's state space. The action a_t corresponds to x_t and the reward r_t to the realized revenue of a potentially sold product at the respective price of price vector x_t . The set of all possible prices defines the problem's action space. Finally, the one-step-transition dynamics p are defined by the product of the arrival probabilities $\lambda_{t,k}$ and the purchase probabilities $q_{t,j}$.

5 Development of policies

In this section, we describe the benchmark and behavior policies (Sect. 5.1) for the network dynamic pricing problem defined in Sect. 4, as well as the critic-regularized regression offline RL algorithm in Sect. 5.2. Finally, in Sect. 5.3 we address important aspects regarding the technical implementation of the algorithms.

5.1 Benchmark and behavior policies

For certain instances, it is computationally tractable to calculate the optimal policy, described in Sect. 5.1.1, which will serve as a benchmark. In Sect. 5.1.2, we describe the two behavior policies that we use to generate the “historic” static data set. The first is based on a systematic randomization of the optimal policy. The idea is that we imitate a suboptimal policy whose deviation from the optimal policy can be controlled. Sect. 5.1.3 contains the formalization of a second behavior policy which is based on a state-of-the-art heuristic from the literature.

5.1.1 Optimal policy (OPT)

The optimal policy (OPT) can be determined by dynamic programming, i.e., it is the solution to

$$V(c_t, t) = \max_{x_t} \left\{ \sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{J}_k} q_{t,j}(x_t) \left(x_{t,j} + V(c_t - M_j, t+1) - V(c_t, t+1) \right) + V(c_t, t+1) \right\} \quad (1a)$$

$$V(0, t) = 0 \quad (1b)$$

$$V(c_t, 0) = 0, \quad (1c)$$

where overbooking is not allowed, meaning that only products j are offered for which $c_t - M_j \geq 0$. Equation (1a) – often denoted as Bellman function – is a recursive formulation of the state value, which quantifies the expected revenue-to-come following the optimal policy. It formalizes that, given a state (c_t, t) , there are multiple

possible outcomes when products are offered at price vector x_t . That is, with probability $q_{t,j}$, a customer arrives and purchases product j , in which case a revenue of $x_{t,j}$ is realized and the state transitions to $(c_t - M_j, t + 1)$. If no customer arrives or a customer arrives and decides not to purchase, the state transitions to $(c_t, t + 1)$. Equations (1b) and (1c) define the boundary conditions, formalizing that the expected revenue to come is zero if there is no capacity left or no time left, respectively.

The optimal price vector x_t^* is the solution of (1a–1c) and is given by

$$x_t^* = \operatorname{argmax}_{x_t} \left\{ \sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{J}_k} q_{t,j}(x_t) \left(x_{t,j} + V(c_t - M_j, t + 1) + V(c_t, t + 1) \right) \right\}. \quad (2)$$

Due to the curses of dimensionality (Powell 2016), computing x_t^* becomes computationally intractable already for presumably small instances. For example, for $T = 100$ periods and $L = 10$ legs with an initial capacity $C = 100$ each, there are $T \cdot C^L = 10^{21}$ states. The fact that not all states are reachable when there is at most one sale per period reduces the state space, but not sufficiently with regard to tractability. The action space for $K = 10$ markets with $|\mathcal{J}_k| = 10$ products each and $Z = 10$ prices results in $K \cdot Z^{|\mathcal{J}_k|} = 10^{11}$ price points each period, such that (2) is not solvable efficiently.

5.1.2 Randomized optimal policy (R-OPT)

For the instances in which OPT is computable, we propose the randomized optimal policy (R-OPT) as behavior policy that deviates from OPT in a randomized, yet systematic manner. The idea is to design a suboptimal behavior policy whose deviation from the optimal one can be controlled. Formally, let $\epsilon_j \sim \mathcal{N}(\mu, \sigma^2)$ be a normally distributed random variable with mean μ and variance σ^2 for product j . Then, the R-OPT price of product j in period t $x_{t,j}$ is determined by projecting $\epsilon_j x_{t,j}^*$ onto the set of discrete prices $\mathcal{F}_j$ where the ϵ_j are independent of one another. With $\mu < 1$ ($\mu > 1$), we can construct a systematically biased policy with too low (high) prices on average. The consistency in pricing can be adjusted by the choice of σ^2 .

5.1.3 Randomized deterministic linear program-based policy (R-DLP)

The second behavior policy, denoted as randomized deterministic linear programming-based policy (R-DLP), builds on a standard approach in RM for solving network dynamic pricing problems heuristically. That is, as the name suggests, it involves solving a deterministic linear program (DLP), as it was, e.g., discussed in Gallego et al. (2004); Liu and Van Ryzin (2008) to solve a network capacity control problem and later in Schön (2010); Erdelyi and Topaloglu (2011); Barz et al. (2023) for dynamic pricing.

The DLP that R-DLP is built upon reads as follows:

$$\max_{w_k^z} \sum_{k \in \mathcal{K}} \sum_{z \in \mathcal{Z}_k} v_k^z w_k^z \quad (3a)$$

s.t.

$$\sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{J}_k} \sum_{z \in \mathcal{Z}_k} m_{j,l} q_j^z w_k^z \leq C_l \quad \forall l \in \mathcal{L} \quad (3b)$$

$$\sum_{z \in \mathcal{Z}_k} w_k^z = \tau \quad \forall k \in \mathcal{K} \quad (3c)$$

$$w_k^z \geq 0 \quad \forall k \in \mathcal{K} \quad (3d)$$

In contrast to the problem statement in Sect. 4 and OPT in Sect. 5.1.1, this formulation does not directly use the price vector x_l as decision variable. Instead, for each market k , it makes use of price lists f_k^z with $z \in \mathcal{Z}_k$, where $\mathcal{Z}_k$ denotes the market-specific price index set of all possible joint prices for all products (not to be confused with the product-specific price index set $\mathcal{Z}_j$). Thus, one price list contains one price for each of the market's product, i.e., $f_k^z = \{f_j^z : j \in \mathcal{J}_k\}$. Further, if prices are set according to price list f_k^z , the purchase probability for product j in market k is q_j^z . Thus, for one period, the expected revenue in market k is $v_k^z = \sum_{j \in \mathcal{J}_k} q_j^z f_j^z$ and the expected revenue across all markets is $\sum_{k \in \mathcal{K}} v_k^z$. The decision variables are the number of time periods price list f_k^z is set in market k , denoted as w_k^z . Constraints (3b) ensure for each leg l that the expected capacity consumption $\sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{J}_k} m_{j,l} q_j^z$ does not exceed capacity C_l . Finally, constraints (3c) ensure that the times for which certain price lists are chosen sum to the remaining time τ .

As standard in these DLP-based approaches, to derive (DLP-)optimal prices x_k^* for each market k , given a certain state (c_l, τ) , the DLP (3a–3d) is solved first. Then, for every leg l , the bid-prices π_l are determined from the optimal dual variables of the corresponding constraint in (3b). Finally, for each market k , the optimal prices are calculated with

$$x_k^* = \operatorname{argmax}_{x_k^z} q_k^z (x_k^z - \sum_{j \in \mathcal{J}_k} \sum_{l \in \mathcal{L}} \pi_l m_{j,l}). \quad (4)$$

This procedure of solving (3a–3d), determining bid-prices π_l and solving (4) is then repeated for each pricing decision, with updated τ – depending on the remaining time and C_l – depending on the remaining capacity after potential purchases. The idea of the procedure is, first, to simplify the problem dynamics as well as the customer choice behavior to obtain the DLP (3a)–(3d) which can be efficiently solved. The number of possible joint prices can be large which implies large number of decision variables. However, the number of constraints in (3a)–(3d) always remains manageable, which is why tractable solution methods based on column generation exist. The second idea is to derive prices with (4) by decoupling the network problem into multiple single-leg problems while accounting for this decomposition

through considering opportunity costs – approximated by the bid-prices – that come with selling a particular product.

Finally, to derive the R-DLP prices, we randomize the prices resulting from the above described DLP-based procedure in the same manner as for R-OPT, i.e., by projecting $\epsilon_j x_{t,j}^*$ onto the set of discrete prices $\mathcal{F}_j$ for every product $j \in \mathcal{J}$, where $\epsilon_j \sim \mathcal{N}(\mu, \sigma^2)$ (see Sect. 5.1.2). The R-DLP parameterization with $\mu = 1, \sigma^2 = 0$ corresponds to the “original” DLP policy as suggested in literature.

5.2 Critic-regularized regression

In this section, we describe and formalize the critic-regularized regression (CRR) offline RL algorithm as proposed by Wang et al. (2020). Section 5.2.1 describes its basics, followed by Sect. 5.2.2, in which we propose an adaptation of the actor loss function which is specifically designed for a particularity of network dynamic pricing. In Sect. 5.2.3, we describe the developed state and action space reformulation to obtain a scalable approach that is suited for the considered problem. We denote the policy obtained from training with the CRR algorithm as CRR policy.

5.2.1 Basic algorithm

The fundamental ideas of CRR have been described in Sect. 2.3. To comprehend the specific algorithmic steps of CRR in the following, an understanding of the mathematical formulation of actor and critic updates is required. Since such description goes beyond the scope of this work, we refer to (Sutton and Barto 2018, Chapter 13.5) as a starting point.

Algorithm 1 Critic-regularized regression (stylized)

Input: Static data set $\mathcal{D}_{static} = \{(s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, s_{t+1}^{(i)})_{i=1, \dots, n}\}$,
 critic (neural network) Q_ϕ with parameter vector ϕ ,
 actor (neural network) π_θ with parameter vector θ

for $n_{updates}$ **do**

1. Sample b transitions from $\mathcal{D}_{static}$, $\{(s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, s_{t+1}^{(i)})\}$
2. Update actor with gradient:

$$\nabla_\theta \left(-\frac{1}{b} \sum_i \log(\pi_\theta(a_t^{(i)} | s_t^{(i)})) g(Q_\phi, \pi_\theta, a_t^{(i)}, s_t^{(i)}) \right)$$
3. Update critic with gradient:

$$\nabla_\phi \left(\frac{1}{b} \sum_i \left(Q_\phi(s_t^{(i)}, a_t^{(i)}) - \left(r_t^{(i)} + \gamma \mathbb{E}_{a \sim \pi_\theta(\cdot | s_{t+1}^{(i)})} [Q_\phi(s_{t+1}^{(i)}, a)] \right) \right)^2 \right)$$

end for

Recall that in the offline RL setting, a static data set of transitions $\mathcal{D}_{static}$ is given (see Sect. 2.3). CRR now iteratively updates the parameter vectors θ and ϕ

of the actor π_θ and the critic Q_ϕ $n_{updates}$ times, respectively. This update consists of the following three steps which are formalized in Algorithm 1:

1. First, a batch of b transitions is sampled from the static data set $\mathcal{D}_{static}$.
2. Next, this batch is used to evaluate the gradient w.r.t. θ . Here, g is a non-negative, scalar function which increases monotonically in Q_ϕ . For example,

$$g := \mathbf{1} \left[Q_\phi(s_t^{(i)}, a_t^{(i)}) > \mathbb{E}_{a \sim \pi_\theta(\cdot | s_{t+1}^{(i)})} [Q_\phi(s_{t+1}^{(i)}, a)] \right]$$

may be chosen, where $\mathbf{1}$ is the indicator function. Intuitively, g acts as a filter function which passes only “good” transitions. More formally, g passes transitions $(s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, s_{t+1}^{(i)})$ for which in state $s_t^{(i)}$ the Q-value of taking the action $a_t^{(i)}$ in state $s_t^{(i)}$ observed in the data is larger than the expected Q-value of choosing an action according the current policy π_θ . Thus, only actions which improve the current policy are used to determine the gradient which is used in the actor’s parameter update. Due to the lack of exploration in offline RL algorithms, the policy improvement is limited to information contained in the support of the data. In other words, favorable state-action combination which were not observed in the static data set cannot be used to improve the policy.

3. Finally, the entire batch (without filtering) is used to update the critic parameter vector. This is done by taking a gradient descent step to minimize the sum of squared errors between the estimated value of the Q-function and its 1-step bootstrap estimate. The next action inside the 1-step bootstrap estimate of the Q-function, a , is taken under the current actor policy π_θ , not under the behavior policy. Further, it is important to note that this does not require any online interaction with the environment.

When comparing CRR to standard standard actor-critic algorithms, three features stand out:

- The update of the critic is identical for both.
- While actor-critic methods interact with the environment after each update to generate new transitions, CRR only resamples from the static data set. This first difference is rather obvious, considering that CRR, in contrast to standard actor-critic algorithms, is an offline RL algorithm.
- The second difference is less obvious, but important: Standard actor-critic algorithms compute the gradient

$$\nabla_\theta \left(\frac{1}{b} \sum_i \log(\pi_\theta(a_t^{(i)} | s_t^{(i)})) Q_\phi(s_t^{(i)}, a_t^{(i)}) \right)$$

to update the policy towards actions with high Q-values and away from actions with low Q-values. CRR, in contrast, computes the gradient

$$\nabla_{\theta} \left(-\frac{1}{b} \sum_i \log \left(\pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \right) g(Q_{\phi}, \pi_{\theta}, a_t^{(i)}, s_t^{(i)}) \right)$$

to update the policy to imitate actions contained in the static data set that are better than the actions proposed by the current policy.

The actor update conducted by CRR is not myopic for two reasons: First, the criterion used to assess whether an action is better than the current policy is based on the Q -value, which – by definition – is not myopic. Second, if a transition is filtered out during one iteration of CRR, it does not mean that it is automatically filtered out for all subsequent iterations. That is, if an action is considered unfavorable relative to some current policy, it will not be used for this immediate update. However, if the same action is favorable under some subsequent current policy, this action will be used for the immediate update.

5.2.2 Adaptation of loss function

The basic CRR approach above requires that every sample i in the static data set $\mathcal{D}_{static}$ is of the same dimension. While this seems to be granted, it is actually not for collected data in network dynamic pricing problems like the one described in Sect. 4. Consider a situation in which one or more products cannot be offered due to a lack of resources. In this case, no prices are set for these unavailable products. Thus, the samples' dimensions differ with regard to $|a_t^i|$. More generally speaking, in network dynamic pricing problems, the action space dimension is state-dependent.

A rather naive solution to this state-dependent action space dimension is to augment the data by using dummy prices (e.g. random prices or last set prices) for the products with depleted resource(s). The issue is that – due to the generalization of the actor π_{θ} neural network – these dummy prices (in states where products are unavailable) cause the actor to falsely learn setting these prices in similar states (where products are available). To address this issue, we propose an adaptation of the basic CRR algorithm that modifies the loss function. More specifically, we remove the products with depleted resource(s) from the loss function when performing the regression on the filtered actions, i.e. in the actor update in Step 2 in Algorithm 1. That is, instead of $\log \left(\pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \right) = \log \left(\prod_{j \in \mathcal{J}} p_{\theta, s_t^{(i)}, j}(a_{t,j}^{(i)}) \right)$, we use $\log \left(\prod_{j \in \mathcal{J}'} p_{\theta, s_t^{(i)}, j}(a_{t,j}^{(i)}) \right)$ where $\mathcal{J}'$ denotes the set of still available products, $a_{t,j}^{(i)}$ the price of product j , and $p_{\theta, s_t^{(i)}, j}$ the probability density function of the Gaussian distribution for product j under the policy parameterization θ , given the current state $s_t^{(i)}$. Note that dummy prices still need to be set to make the implementation work, but that these dummy prices no further effect the training. This adaptation from the basic CRR algorithm turned out to be beneficial for the performance of the CRR policy, especially for larger instances where a substantial proportion of products' resources become depleted over time.

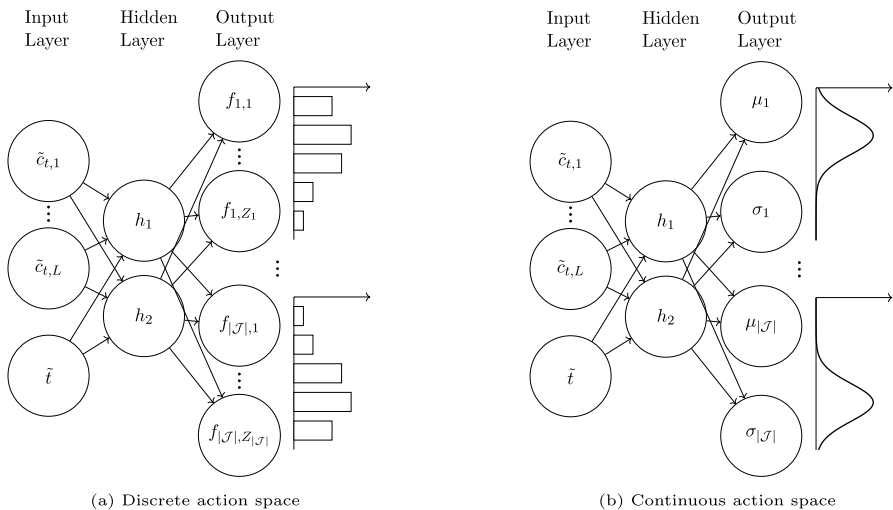


Fig. 4 Schematic visualization of the action space reformulation: For each product j , there are Z_j admissible price points (in each market). With the discrete action space, the actor assigns a probability to each of the products' admissible price points. Thus, the output layer's dimension is equal to the number of products multiplied by the number of admissible price points. With the continuous action space, the actor specifies for each product a continuous parametric distribution over continuous prices. Utilizing normal distributions, this requires μ_j and σ_j for each product, reducing the output layer's dimension to two times the number of products. Prices sampled from the continuous distributions are mapped back to the admissible price points in a later step (not displayed)

5.2.3 State and action space reformulation

In the following, we first discuss why the straightforward application of the CRR described in the previous subsection to the generic network dynamic pricing problem described in Sect. 4 does not scale to real-life instances. To address this problem, we then develop reformulations of action and state space that ultimately result in a scalable approach.

- Straightforward application:** The actor π_θ is a function which maps the state space $\mathcal{S}$ to the action space $\mathcal{A}$. The critic Q_ϕ maps each possible combination of state and action, i.e., $\mathcal{S} \times \mathcal{A}$, to its corresponding value estimate. In the considered problem, the action space consists of all possible combinations of prices $x_t = (x_{t,j} : j \in \mathcal{J}_k, k \in \mathcal{K})^T$ for all products j in all markets k across all periods t , where a product-specific price $x_{t,j}$ has to be chosen from the respective set of admissible price points $\mathcal{F}_j$. The state space consists of all possible combinations of period t and the capacity vector $c_t = (c_{t,1}, c_{t,2}, \dots, c_{t,L})$ with its entries for each product l . In a straightforward application, a one-hot encoding is used to represent this discrete state space. Thus, the actor has $|\mathcal{S}|$ input nodes and $|\mathcal{A}|$ output nodes, where each of them yields the probability with which the corresponding action would be taken. Thus, the critic with one-hot encoded state and action space has $|\mathcal{S}| + |\mathcal{A}|$ input nodes. This straightforward application does not scale to real-life instances, because of the exponentially

(w.r.t. the number of products) growing action space. The state space “only” grows linearly (w.r.t. the number of resources), but still grows too large for real-life instances, even if the action space size was no issue. On the one hand, the optimization of the neural network becomes intractable, due to the high number of parameters. On the other hand, even if this was no problem, the data in reasonably many observations is too sparse to allow the networks to generalize well.

- *Action space reformulation:* We propose a low-dimensional reformulation of the action space, as schematically depicted in Fig. 4. Specifically, we propose to replace the discrete action space with a continuous one, in which the action space distribution is parameterized as follows. For each product j , we define a random variable $\tilde{x}_j$ which follows a normal distribution, i.e., $\tilde{x}_j \sim \mathcal{N}(\mu_j, \sigma_j^2)$, as it is common for continuous action spaces (Sutton and Barto 2018). Selecting an action for a given state is done by sampling all prices $\tilde{x}_j$ from these normal distributions, followed by projecting them onto the original admissible discrete prices $\mathcal{F}_j$. The mean μ_j and the standard deviation σ_j of every product j now become the output nodes of the actor. Thus, there are only two output nodes for every product and the actor with its $2|\mathcal{J}|$ output nodes grows linearly in the instance size and, thus, remains relatively small even for large instances. For the action-inputs of the critic, instead of using a one-hot encoding of the discrete action space, we use one input per product (for each market) that corresponds to the (normalized to the interval $[-1, 1]$) price set for the respective product. Thus, there are only $|\mathcal{J}|$ action-inputs for the critic.
- *State space reformulation:* As for the action space, we propose a low-dimensional reformulation of the state space by replacing the discrete space with a continuous one. Specifically, we define continuous variables $\tilde{c}_{t,l}$ for each resource l as well as $\tilde{t}$ for the time. These variables represent the fractions $\tilde{c}_{t,l} = c_{t,l}/C_l$ of the still available capacities $c_{t,l}$, and the remaining time $\tilde{t} = t/T$, respectively. This results in a strongly reduced state space representation with $L + 1$ states, with corresponding few input nodes of actor and critic.

In summary, the presented state and action space reformulations result in low-dimensional representations which require comparably small neural networks for actor and critic. Further, the state-space reduction results in denser data, and allows the CRR to generalize easier. Overall, the CRR algorithm becomes scalable also to real-life size instances.

As shown in the literature review in Sect. 3, none of the closest related works that consider RL-based dynamic pricing consider a *network* problem, presumably due to the scalability issues described above. The presented state and action space reformulations allow for scalability, regardless of the specific algorithm. Thus, these reformulations also enable further work on online RL in the context of dynamic pricing.

5.3 Implementation

We implemented the environment and algorithms in Python 3.8. To solve the DLP (Sect. 5.1.3), we used Gurobi and, more specifically, the library `gurobipy` 10.0.23. For the CRR, we built on the open source library `RLlib` (Liang et al. 2018, specifically Ray 2.2.0), which provides implementations of numerous on- and offline RL algorithms, using `torch` 1.13.1 as framework for the neural networks. To model the environment, we used OpenAI's open source library `Gym` 0.23 (Greg et al. 2016).

6 Computational study

In Sect. 6.1, we present the construction of three different problem instances, denoted as “small”, “medium”, and “large” instance. For each of them, we conduct a computational study whose setup we elaborate in Sect. 6.2. In Sect. 6.3, we discuss an issue related to sparsity of “good” actions in large instances and present our setup adaption. Finally, in Sect. 6.4, we present the corresponding results.

6.1 Instances

Based on the literature, we construct three instances that primarily vary in their size. Their network structures are depicted in Fig. 5. The instances' parameters are summarized in Table 2. In the following, we describe the large instance in more detail and briefly describe the simplifications and changes necessary to obtain the medium and small instance. The instances are composed of instance components from literature, because the problem at hand is a generalization (w.r.t. resources and products) of problems discussed in literature (see Sect. 4).

Regarding the network structure, the network of the large instance was proposed in the seminal work on network dynamic pricing by Gallego and van Ryzin (1997). As depicted in Fig. 5c, it consists of six nodes, of which two are hubs (nodes 2 and 3). There are $L = 11$ legs which each represent one resource. The resources' initial capacities are displayed in the last row of Table 2, and they vary in the range between $C_l = 4$ and $C_l = 16$, depending on the scenario. Note that these initial capacities are smaller than in Gallego and van Ryzin (1997), because we consider individual customer choices with at most one purchase per period, while in the seminal work demand is modeled on an aggregate level through a demand function which can result in several purchases per period.

With regard to markets and products, there are $K = 18$ markets with two products each. The entries $m_{j,l}$ of the resource consumption matrix M which are also stated in Table 2 define which product consume which resource(s). For example, product $j = 5$ consumes one capacity unit each of resource $l = 1$ and $l = 5$. This matrix is also taken from Gallego and van Ryzin (1997). However, in contrast to their work in which customer choice is not considered, our problem formulation allows for multiple products per market (see Sect. 4) and we define two products per market between which customers can choose. The (time-homogeneous)

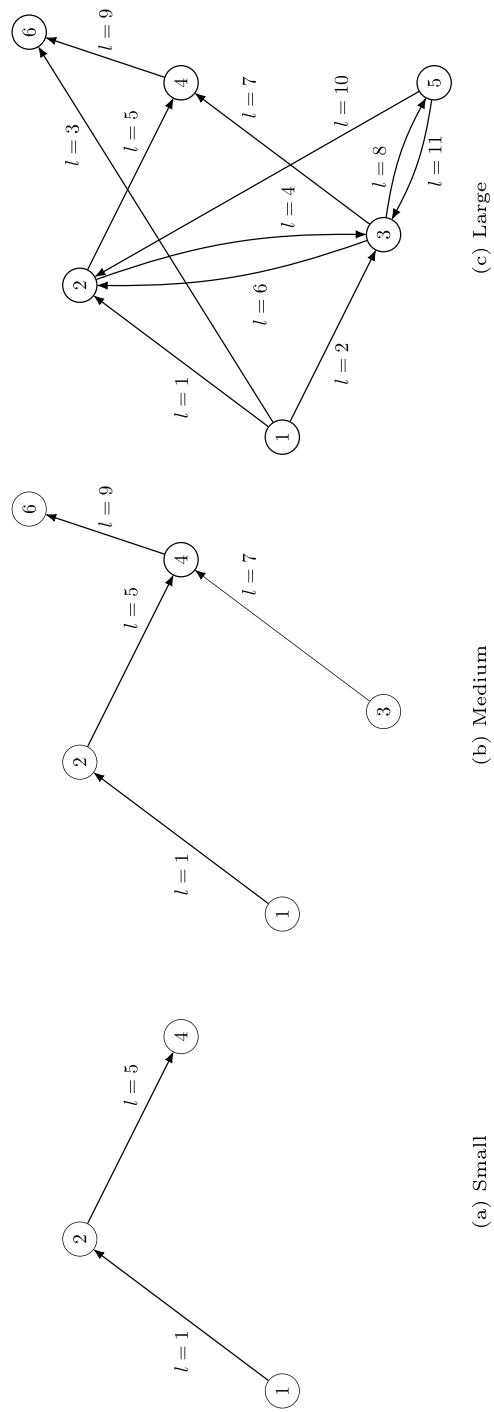


Fig. 5 Network structures of the three instances. The network of the large instance was proposed in Gallego and van Ryzin (1997)

arrival probabilities $\lambda_{t,k}$ for a customer of market k is displayed in the second column of Table 2. The respective residual defines the no choice probability $\lambda_{t,0}$. To obtain reasonable instances with regard to scarcity of the resources, we use fewer periods compared to Gallego and van Ryzin (1997), such that they match the initial capacities (see above).

Concerning the customer choice behavior, we adopt model and parameters from Talluri and van Ryzin (2004b) and Schön (2010). That is, we also assume a multinomial logit (MNL) and we assume the identical exponential utility value function $u_{t,j} = \exp(\beta_j x_{t,j})$ which only depends on the product-specific (time-homogeneous) parameter β_j as well as on the set price $x_{t,j} \in \mathcal{X}_j$ with a (time-homogeneous) utility for the no-choice option u_0 , the purchase probability of product j according to the MNL and given that a customer of market k arrived is $q_{t,j} = u_{t,j} / (u_0 + \sum_{i \in \mathcal{J}_k} u_{t,i}) \lambda_{t,k}$. The β_j and u_0 are chosen slightly different from the above named two works, to allow prices to have a higher impact on the choice probabilities, because we found the originally resulting customer decisions in the admissible price range to be somewhat single-sided (Fig. 6a and b), compared to those for the parameters chosen in our work (Fig. 6c and d). The corresponding parameters are summarized in Table 2 as well, and the no-choice attraction value is chosen to $u_0 = 0.1$ in all instances. Neither the customer arrival probability nor the customer choice model parameters are known to the CRR algorithm. These

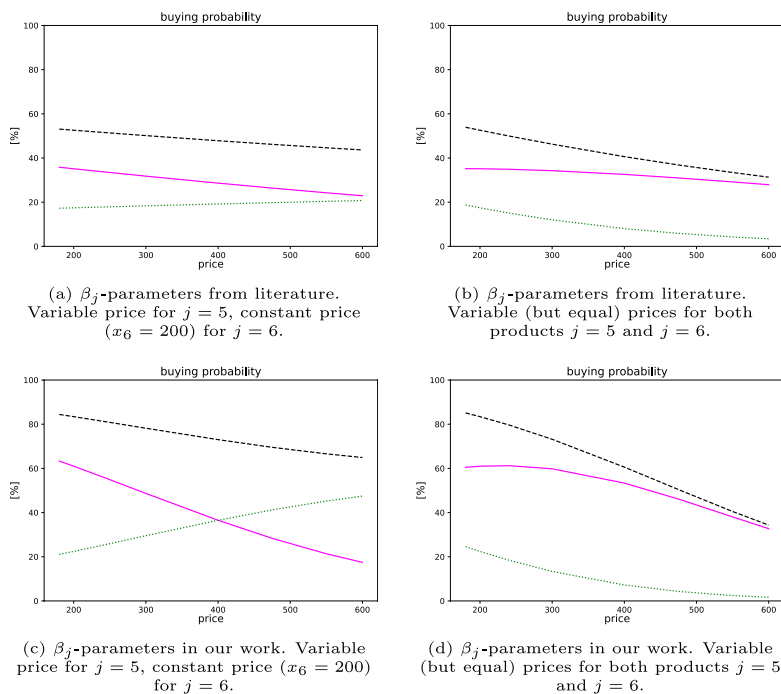


Fig. 6 Exemplary probabilities for buying product $j = 5$ (magenta), $j = 6$ (green), and their sum (black) for different parameterizations of β_j and prices

Table 2 Parameters for arrival probabilities, resource consumption, choice model, and capacities

Market k	Arrival prob. $\lambda_{k,l}$			Product j	Resource consumption $m_{j,l}$ of resource l											β_j in instance			
	small	medium	large		1	2	3	4	5	6	7	8	9	10	11	small	medium	large	
1	0.3	0.15	0.05	1	1	0	0	0	0	0	0	0	0	0	0	-0.01	-0.01	-0.01	
2			1	0	0	0	0	0	0	0	0	0	0	0	0	-0.02	-0.02	-0.02	
3			0	1	0	0	0	0	0	0	0	0	0	0	0			-0.01	
4			0	1	0	0	0	0	0	0	0	0	0	0	0	0			-0.02
3	0.3	0.15	0.05	5	1	0	0	0	1	0	0	0	0	0	0	-0.005	-0.005	-0.005	
6			1	0	0	0	1	0	0	0	0	0	0	0	0	-0.01	-0.01	-0.01	
7			0	1	0	0	0	0	0	0	1	0	0	0	0			-0.005	
8			0	1	0	0	0	0	0	0	1	0	0	0	0	0			-0.01
5			0.05	9	0	0	1	0	0	0	0	0	0	0	0			-0.01	
10			0	0	1	0	0	0	0	0	0	0	0	0	0			-0.02	
11			0	0	0	1	0	0	0	0	0	0	0	0	0	0			-0.01
12			0	0	0	1	0	0	0	0	0	0	0	0	0	0			-0.02
7	0.3	0.15	0.05	13	0	0	0	0	1	0	0	0	0	0	0	-0.01	-0.01	-0.01	
14			0	0	0	0	1	0	0	0	0	0	0	0	0	-0.02	-0.02	-0.02	
15			0	0	0	1	0	0	0	0	1	0	0	0	0			-0.005	
16			0	0	0	1	0	0	0	0	1	0	0	0	0			-0.01	
9		0.15	0.05	17	0	0	0	0	1	0	0	0	1	0	0	-0.005	-0.005	-0.005	
18			0	0	0	0	1	0	0	0	1	0	0	0	0	-0.01	-0.01	-0.01	
19			0	0	0	0	0	1	0	0	0	0	0	0	0			-0.01	
20			0	0	0	0	0	1	0	0	0	0	0	0	0			-0.02	
11			0.05	21	0	0	0	0	0	0	1	0	0	0	0			-0.01	
22			0	0	0	0	0	0	1	0	0	0	0	0	0			-0.02	

Table 2 (continued)

Market k	Arrival prob. $\lambda_{k,l}$			Product j	Resource consumption $m_{j,l}$ of resource l											β_j in instance		
	small	medium	large		1	2	3	4	5	6	7	8	9	10	11	small	medium	large
12			0.05	23	0	0	0	0	0	0	0	1	0	0	0			-0.01
				24	0	0	0	0	0	0	0	1	0	0	0			-0.02
13		0.15	0.05	25	0	0	0	0	0	0	1	0	1	0	0		-0.005	-0.005
				26	0	0	0	0	0	0	1	0	1	0	0		-0.01	-0.01
14		0.15	0.05	27	0	0	0	0	0	0	0	0	1	0	0		-0.01	-0.01
				28	0	0	0	0	0	0	0	0	1	0	0		-0.02	-0.02
15			0.05	29	0	0	0	0	0	0	0	0	0	1	0		-0.01	-0.01
				30	0	0	0	0	0	0	0	0	0	1	0		-0.02	-0.02
16			0.05	31	0	0	0	0	0	0	0	0	0	0	1		-0.01	-0.01
				32	0	0	0	0	0	0	0	0	0	0	1		-0.02	-0.02
17			0.05	33	0	0	0	0	0	0	1	0	0	0	1		-0.005	-0.005
				34	0	0	0	0	0	0	1	0	0	0	1		-0.01	-0.01
18			0.05	35	0	0	0	0	0	0	1	0	1	0	1		-0.005	-0.005
				36	0	0	0	0	0	0	1	0	1	0	1		-0.01	-0.01
C_j in instance	small				12				16							T in instance		
	medium				12				16		12		12			small	medium	50
	large				12	4	4	8	16	8	12	4	12	4	4	large	large	100 300

Table 3 Discrete prices for all products j . Price point $f_{j,0}$ is the reservation price for which there is no demand

Price index z	1	2	3	4	5	6	7	8	9
Price point $f_{j,z}$	180	200	240	280	300	400	475	550	600

parameters are solely used to calibrate the simulated environment with which the static data set is generated and for evaluating the policies.

Regarding the construction of the small and medium instances, please note first that their network structures, markets and corresponding products, are subsets of the large instance's one (Fig. 5, Table 2). Note further that the small instance still has the characteristic of a network problem (in contrast to two independent single-leg problems), because the two products of market $k = 3$ consume resources of both legs $l = 1$ and $l = 5$, which correspond to the resources of the products of markets $k = 1$ and $k = 3$, respectively. The number of periods were chosen to $T = 100$ and arrival probabilities as well as the MNL model parameters were adapted accordingly to obtain reasonable instances with regard to scarcity.

Finally, regarding prices, we use the same discrete prices as in Talluri and van Ryzin (2004b) and Schön (2010), listed in Table 3.

6.2 Setup

We implement an environment which serves for generating “historic” data as well as for evaluating policies in the dynamic pricing problem described in Sect. 4, including the state and actions space reformulation as described in Sect. 5.2.3. Since we consider an offline RL framework, this environment is *not* used for training the CRR policy.

Table 4 gives an overview on which policies are used in which instance. The optimal policy (OPT), and thus R-OPT, can only be determined for the small instance. For the behavior policies R-OPT and R-DLP, each combination of the indicated sets of μ and σ (Cartesian product) yields one specific parameterized behavior policy. Using $\mu = 1.0$ and $\sigma = 0.0$ for R-OPT equates OPT. As explained in Sect. 5.1, $\mu < 1$ ($\mu > 1$) simulates a decision maker's policy with bias from OPT and DLP towards too low (high) prices. The higher σ , the more variation there is in the behavior policy, simulating that the decision maker was less consistent in the pricing. The reason for testing only few parameter combinations for the medium and large instance is that generating data with the DLP policy is time-consuming, in particular when considering that a lot more runs are required in the larger settings to obtain a comparable density of state-action observations. Note that the limitation is not the training of the CRR policy, but it is a data availability problem.

For every instance and specific parameterized behavior policy, the following four steps, which are depicted in Fig. 7, are executed.

1. *Generate data with behavior policy:* In the first step, data is generated with the specific parameterized behavior policies, e.g. R-OPT with $\mu = 0.8$ and $\sigma = 0.1$.

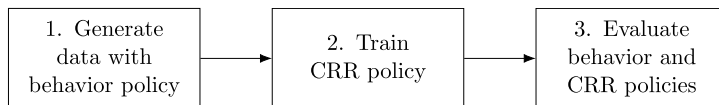


Fig. 7 Schematic setup of the computational study

Table 4 Policies and their parameterizations in the three instances

Policy	Instance		
	small	medium	large
OPT	✓	✗	✗
R-OPT	$\mu \in \{0.6, 0.8, 1.0, 1.2, 1.4\}$ $\sigma \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$	✗	✗
R-DLP	$\mu \in \{0.6, 0.8, 1.0, 1.2, 1.4\}$ $\sigma \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\}$	$\mu \in \{0.7, 1.0, 1.3\}$ $\sigma \in \{0, 0.3, 0.5\}$	$\mu \in \{1.0\}$ $\sigma \in \{0, 0.3\}$

2. *Train CRR policy*: Based on this generated data set, the CRR policy is trained. Depending on the instance size, between 5,000 and 20,000 trajectories were used.
3. *Evaluate behavior and CRR policies*: In the fourth step, the policies are evaluated. The performance of the behavior policy can directly be read from the generated data. To evaluate the performance of the CRR policy, it interacts without exploration with the environment (online evaluation, see Sect. 2.3), depending on the instance size between 300 and 1,000 trajectories.

Finally, we use different visualizations and metrics to analyze the results. Most importantly, we compare the generated revenue which reflects the problem's objective. In this context, we put particular attention on the impact of μ and σ , to assess whether the potential of applying offline RL depends on characteristics of the behavior policy. Besides that, e.g., we compare the trained CRR policy with the behavior policies (and OPT) by considering the prices set in a state.

6.3 Overcoming sparsity of “good” actions

As described in Sects. 5.1.2 and 5.1.3, we randomize the actions of the behavior policies R-OPT and R-DLP by using an *independent* normal distribution for each price. Offline RL requires the existence of “good” actions in the static data set (see Sect. 2.3), but this independent randomization may – in large instances – lead to few such actions in the static data set. For example, if the behavior policy overall has a bias towards too low prices (e.g. R-OPT with $\mu < 1$) and each price now is randomized independently around a too low price, it is very unlikely that *all* prices are higher in the same sample. However, all prices being higher is exactly what would be required to obtain a “good” action. This issue certainly increases for larger instances.

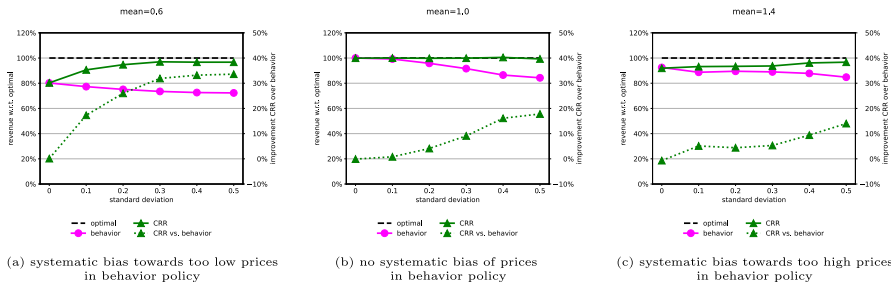


Fig. 8 Average revenue with respect to optimal policy (left axis) and relative improvement of CRR over behavior policy (right axis) for different parameterizations of the **R-OPT** behavior policy

To address this issue, for the large instance, we added a correlation of 0.8 between the randomized prices for the two products of every market. This adaptation turned out to be beneficial. Note that these positive correlations between price changes of products that belong to the same market are reasonable in practice. In particular, it would be irrational to make a premium product cheaper than a standard product of the same market. The specific value 0.8 used was identified by testing values in the range of 0.5 to 1.0 and turned out to strike a balance between too high correlation and a too low one which does not adequately address the issue.

6.4 Results

In this section, we present the numerical results, organizing them according to the instance size introduced in the previous section. That is, the results for the small setting – which our presentation focusses on – are presented in Sect. 6.4.1, followed by high-level summaries of the results for the medium and large settings in Sect. 6.4.2. Note that the reported figures like rewards or remaining capacities always refer to the *average* values resulting from the evaluation.

6.4.1 Small instance

We begin by describing the performance of the CCR policy in comparison to the optimal benchmark and to the behavior policy that led to the data. We first consider R-OPT as behavior policy (see Sect. 5.1.2). Figures 8 and 9 show the respective results for a subset of the parameterizations. Table 5 summarizes the results for all parameterizations. Remember that mean μ and standard deviation σ influence the behavior policy's systematic bias and variation of prices, respectively.

The performance of the CRR policy with R-OPT as behavior policy can be summarized as follows:

- *CRR dominates the behavior policy:* For all combinations of μ and σ , CRR (green solid line) realizes an average revenue which is at least as high as the behavior policy's (magenta solid line) one. For the depicted parameterizations,

the improvement of CRR over the behavior policy (green dotted line) is up to around 30% (Fig. 8a, right axis). The fact that CRR never performs worse than the behavior policy is important, because it means that applying the CRR policy is a cost-free approach with exclusively upside potential. In other words, in the worst case, CRR clones the behavior policy and the realized reward is identical to the one realized in the past. Naturally, the average revenue obtained with the optimal policy (black dashed line) is an upper bound.

- *CRR's improvement depends on the performance of the behavior policy:* Naturally, the improvement depends on the potential that the behavior policy leaves for improvement. For example, for μ close to $\mu = 1.0$ and small σ (Fig. 9a), the behavior policy is close to the optimal policy. The further μ departs from $\mu = 1.0$ and the higher σ , the worse the behavior policy becomes and the more potential it leaves for improvement for CRR (Fig. 9c). Note that relative improvement of CRR over the behavior policy (right axis) is bounded by the difference in average revenue between the optimal and the behavior policy, divided by the average revenue of the behavior policy.
- *CRR improves as variation increases:* Fig. 8 shows the trend that, for all μ , the improvement of CRR over the behavior policy increases with increasing σ . Figure 8a shows that CRR converges towards the optimal policy as σ increases. The same result can be observed from comparing Fig. 9a and c. Conversely, when there is no variation in the behavior policy, CRR is not capable to improve (Fig. 9a) – a well-known fact which holds for all offline RL algorithms. The reason is that, for any given μ , the higher σ , the broader the support of states and actions in the static data set's trajectories. Since “offline RL tries to find the best policy π that is supported by the static data set” (see Sect. 2.3), it is more likely that the static data set contains “good” actions which CRR can learn from.
- *CRR's improvement depends on the behavior policy's systematic bias:* CRR's improvement is higher when the behavior policy has a bias towards low prices (Fig. 8a) than for a bias towards high prices (Fig. 8c). One reason is that the behavior policy performs better when prices have a bias towards high prices. That is, Fig. 12 depicts that the prices resulting from the DLP policy (before randomization) are lower than the optimal ones. Thus, the randomization with a bias towards high prices results in a better behavior policy.

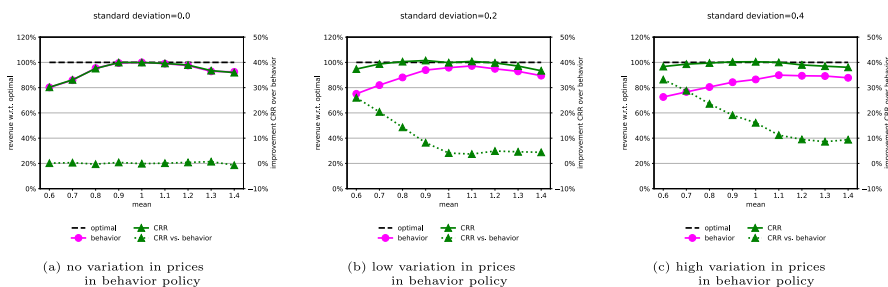


Fig. 9 Average revenue with respect to optimal policy (left axis) and relative improvement of CRR over behavior policy (right axis) for different parameterizations of the **R-OPT** behavior policy

Table 5 Summary of results for small instance with R-OPT as behavior policy

Mean μ	0.8										1.0					1.2					1.4										
	0.6	0.0	0.1	0.2	0.3	0.4	0.5	0.0	0.1	0.2	0.3	0.4	0.5	0.0	0.1	0.2	0.3	0.4	0.5	0.0	0.1	0.2	0.3	0.4	0.5						
Std. dev. σ																															
Revenue	80%	91%	95%	97%	97%	97%	97%	95%	98%	101%	100%	100%	100%	98%	100%	100%	100%	100%	101%	99%	98%	100%	99%	98%	98%	92%	93%	94%	96%	97%	
CCR																															
writ.																															
opti-																															
mal																															
Impr.	0%	17%	26%	32%	33%	34%	34%	-0%	6%	14%	20%	24%	27%	-0%	1%	4%	9%	16%	18%	0%	2%	5%	7%	10%	15%	-1%	5%	4%	5%	9%	14%
CCR																															
over																															
beh.																															

Table 6 Summary of results for small instance with **R-DLP** as behavior policy

	Mean μ										Std. dev. σ									
	0.6	0.7	0.8	0.9	1.0	1.1	1.2	1.3	1.4	1.5	0.6	0.7	0.8	0.9	1.0	1.1	1.2	1.3	1.4	1.5
Revenue	63%	76%	84%	89%	92%	94%	96%	98%	99%	100%	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
CRR	63%	76%	84%	89%	92%	94%	96%	98%	99%	100%	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9
w.r.t. opti-mal																				
Impr. CRR over beh.	3%	23%	32%	39%	42%	46%	50%	54%	58%	62%	2%	26%	32%	35%	38%	39%	40%	41%	42%	43%

The results for the case that the behavior policy is R-DLP (see Sect. 5.1.3), also in comparison to R-OPT as behavior policy, can be summarized as follows (see Figs. 10, 11 and Table 6): In principle, all of the observations from above also hold when the behavior policy is R-DLP. Most importantly, we point out that CRR again dominates the behavior policy for all combinations of μ and σ in terms of realized average reward. The improvements are in the range of up to 42%. Note that, different from above, since R-DLP is a heuristic, the behavior policy for $\mu = 1.0$ and $\sigma = 0$ is suboptimal. As a consequence, the improvement of CRR for $\mu = 1.0$ and $\sigma > 0$ is higher (approximately 10 percentage points) compared to the respective parameterization with R-OPT as behavior policy.

In the following, we analyze the learned CRR pricing policy in more detail. To this end, we exemplarily consider the parameterization in which CRR was trained on the R-DLP behavior policy with $\mu = 1.0$ and $\sigma = 0.4$. Figure 12 depicts the average set prices for certain products by the optimal, the behavior (depicted for $\sigma = 0.0$ to make the visualization easier to grasp without high fluctuations of prices), and the CRR policy, in dependence of (a subset of) the state. For example, Fig. 12a depicts the prices f_1 , i.e., the price for product $j = 1$, in dependence of the time and the capacity c_1 , which is the resource that product $j = 1$ consumes (Table 2). The averages are taken over the respective other capacity that is not displayed, here c_2 . The following observations can be made:

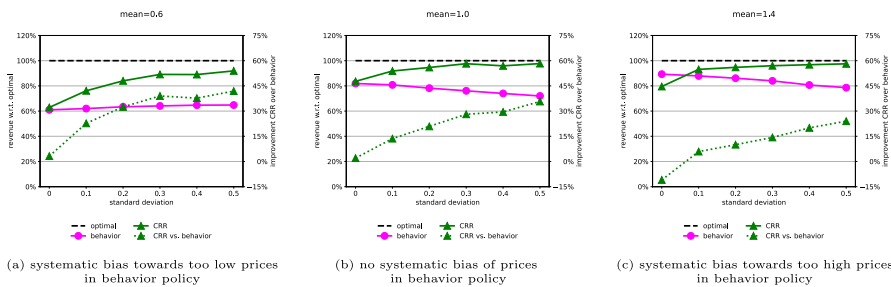


Fig. 10 Average revenue with respect to optimal policy (left axis) and relative improvement of CRR over behavior policy (right axis) for different parameterizations of the **R-DLP** behavior policy

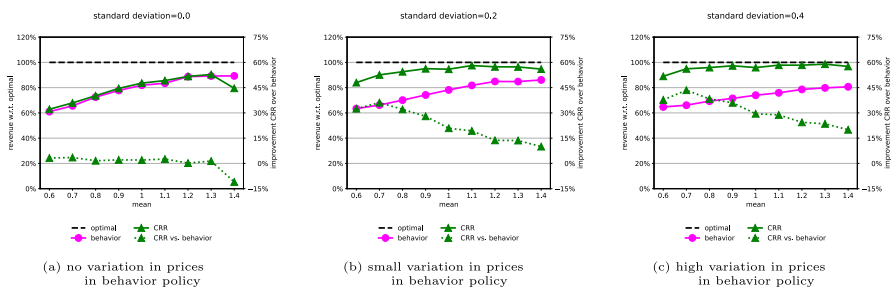


Fig. 11 Average revenue with respect to optimal policy (left axis) and relative improvement of CRR over behavior policy (right axis) for different parameterizations of the **R-DLP** behavior policy

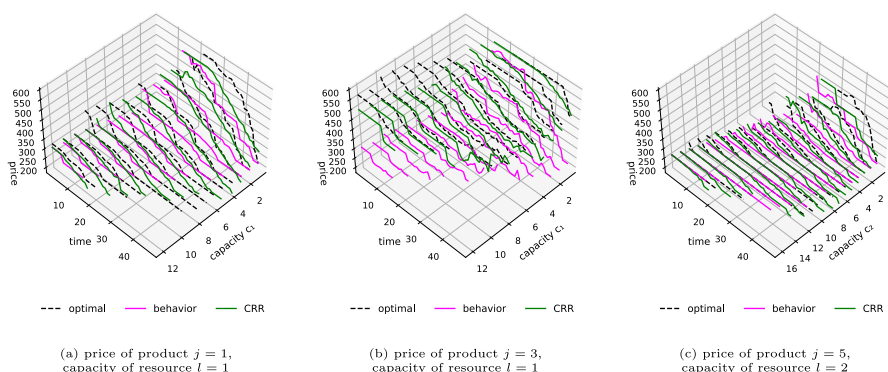


Fig. 12 Comparison of policies: Average prices for different products (e.g. $j = 1$ in subfigure (a)) in dependence of one capacity (e.g. c_2 in subfigure (b)) and the time of the selling horizon. Averages are taken over the capacity not displayed (e.g. c_1 in subfigure (c))

- *CRR's pricing structure is reasonable*: It can be observed that the structure of the CRR policy is reasonable, meaning that monotonicities which are known to exist for the optimal policy also hold for CRR. That is, for a given capacity, prices decrease as time approaches the end of the selling horizon. Further, for a given time, prices have the tendency to increase when capacity becomes scarce.
- *CRR's prices are closer to the optimal policy*: CRR generally, i.e., for each product and for any capacity-time combination, sets higher prices than the behavior policy and, thus, prices are closer to the optimal prices. This is especially true for price f_5 , which is the price of product $j = 5$ which consumes capacities of both resources (see Table 2). Thus, CRR successfully learns that selling this product comes with high opportunity costs and in turn sets higher prices.

Finally, for the same parameterization as considered previously ($\mu = 1.0$ and $\sigma = 0$), we consider results of the realized revenue and the capacity consumption by the three policies in more detail. In particular, in Fig. 13a, we consider the average reward realized over the course of the selling horizon. In Fig. 13b, the respective average cumulative reward is depicted. Figure 14a and b show the average capacity for the different periods of the selling horizon. The observations are as follows:

- *Consistent dominance of CRR*: Figure 13b shows that CRR outperforms the behavior policy from the beginning of the selling horizon on, indicating that it takes superior actions already at the beginning of the selling horizon. During the first four-fifths of the selling horizon, CRR even outperforms the optimal policy but this advantage is overcompensated by the optimal policy through substantially higher rewards in the last fifth (Figure 13a).
- *CRR outperforms in particular at end of selling horizon*: Considering both Figs. 13 and 14 in combination allows to derive that the improvement of CRR over the behavior policy is mainly related to the additional revenue realized during the last periods of the selling horizon. That is, the higher rewards of CRR

in the last periods (Fig. 13a) result in an increasing difference of the cumulated rewards between CRR and the behavior policy (Fig. 13a). Both Fig. 14a and b indicate that the behavior policy has few or even no capacity left during these last periods, because prices have been too low throughout the selling horizon, as described above. Similarly, this holds for the comparison between the optimal policy and CRR, meaning that the optimal policy outperforms CRR in particular in the last periods.

6.4.2 Medium and large instances

Table 7 summarizes the results for all parameterizations in the medium and large instance. Note that the optimal policy cannot be computed for these instances (see Sect. 5.1.1). The results can be summarized as follows:

- *CRR outperforms the behavior policy:* As for the small instance, CRR consistently realizes higher revenues than the R-DLP behavior policy. For the tested

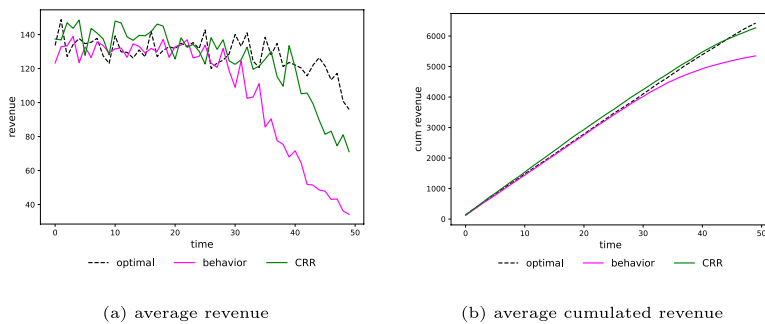


Fig. 13 Average revenues realized over the course of the selling horizon by the three policies

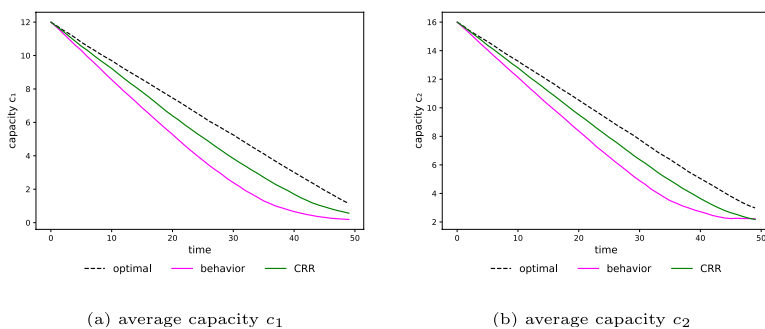


Fig. 14 Average capacities over the course of the selling horizon by the three policies

Table 7 Summary of results for medium and large instance

Instance	medium									large
	0.7			1.0			1.3			1.0
Mean μ										
Std. dev. σ	0.0	0.3	0.5	0.0	0.3	0.5	0.0	0.3	0.5	0.3
Revenue CRR w.r.t. R-DLP $\mu = 1.0$, $\sigma = 0.0$	79%	97%	98%	102%	115%	118%	116%	123%	125%	108%
Impr. CRR over beh.	5%	30%	34%	2%	28%	41%	1%	16%	31%	21%

parameterizations, the improvement lies in the range of up to 34%. With the exception of $\mu = 1.4$ and $\sigma = 0.0$, all learned CRR policies yield a higher revenue than the corresponding behavior policy.

- *CRR scales to large instances:* That CRR can efficiently be computed in the first place also for the medium and large instances is an important fact, as it demonstrates that it scales to more realistic instances. The fewer parameterizations for these instances are due to the amount of time needed to generate the data from the behavior policy. In particular, solving the optimization problem of R-DLP (see Sect. 5.1.3) for every period is computationally demanding.

6.5 Robustness Analysis

To assess the robustness of the proposed CRR approach in the context of network dynamic pricing, we perform additional computational experiments for which we alter the following key components. First, we modify the network dynamic pricing problem with regard to the available information at decision making time. Second, we use instances that differ in the network structure and in their size. Third, we use a different behavior policy to generate the static data set. Thus, compared to above, we evaluate the performance of the proposed approach under very different circumstances. In the following sections, we describe the modified problem (Sect. 6.5.1), the new instances (Sect. 6.5.2), and the new behavior policy (Sect. 6.5.3). Section 6.5.4 contains the corresponding results.

6.5.1 Modified Problem Statement

In the previously investigated problem described in Sect. 4, the provider was unable to observe whether a customer arrived and, if so, on which market. Instead, at the beginning of each period the provider sets the prices for all products of all markets and subsequently only observes whether and, if so, which product was purchased. Thus, if no product was purchased, the provider cannot differentiate between the two possibilities that no customer has arrived or that a customer arrived but decided not to purchase.

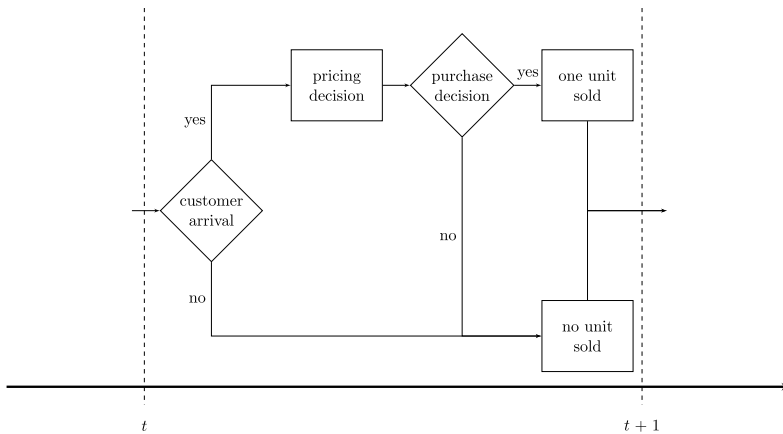


Fig. 15 Schematic depiction of the modified sales process. Each period consists of potentially three subsequent events, i.e., (potential) customer arrival, (potential) pricing decision, and (potential) purchase decision. The provider is able to observe customer arrival and purchase decisions

The pricing problem considered in this section is modified. That is, the provider can now observe whether a customer arrives and, if so, on which specific market. The modified sales process, which is schematically depicted in Fig. 15, is as follows: At the beginning of each period t , with probability $\lambda_{t,k}$ a customer who is interested in buying a product of market k arrives and with probability $\lambda_{t,0}$ no customer arrives. Given a customer arrived on market k , the provider sets prices for the requested market – more precisely, for the requested resource consumption. Subsequently, the customer decides whether to purchase one of the products of that market at the prices set by the provider. Thus, compared to the problem described in Sect. 4, the provider has full observability.

6.5.2 Modified instance

The largest previously studied network instance described in Sect. 6.1 was the custom network of Gallego and van Ryzin (1997). In contrast, here, we use a single hub and N spoke network, which is widely used in the literature (see e.g. Adelman (2007), Bertsimas and De Boer (2005), Zhang and Adelman (2009)). We consider an instance with $N = 12$ spokes, 24 legs (each with an initial capacity of 10), 168 markets (i.e. origin-destination pairs) and 336 products, two in each market (the specific network used is depicted in Appendix A). In terms of size, this network is comparable to the largest in the literature (see Appendix C for a tabular overview of instances), which fully disclose their network structure, e.g. Bertsimas and De Boer (2005), Zhang and Adelman (2009) and Adelman (2007) with 240, 272 and 420 products, respectively. Note that all these studies also analyze single hub and N spoke networks. There are also studies that use larger networks (e.g. Perakis and Roels (2010), Van Ryzin and Vulcano (2008) and Birbil et al. (2014)), however, these do not fully disclose their network structure so we cannot replicate and use

them for our analyses. For the market arrival probabilities $\lambda_{t,k}$, we sampled from a uniform distribution within the interval $[0, 0.01]$ for each market, resulting in a cumulative arrival rate of 0.82.

6.5.3 Modified behavior policy: randomized decomposition by revenue allocation

The third behavior policy, denoted as randomized decomposition by revenue allocation policy (R-DRA) builds on an approach by Erdelyi and Topaloglu (2011). Its central idea is to reformulate the problem such that it can be decomposed into several sub problems – one for each leg – which then are independently solvable in a computationally efficient manner. It augments the DLP formulation (3a)–(3d) by introducing copies of the decision variables w_k^z for every leg l , resulting in $w_{l,k}^z$ (used in the constraints) and $w_{\psi,k}^z$ (used in the objective function). The corresponding linear program is

$$\max_{w_{\psi,k}^z} \sum_{k \in \mathcal{K}} \sum_{z \in \mathcal{Z}_K} v_k^z w_{\psi,k}^z \quad (5a)$$

s.t.

$$\sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{J}_k} \sum_{z \in \mathcal{Z}_k} m_{j,l} q_j^z w_{l,k}^z \leq C_l \quad \forall l \in \mathcal{L} \quad (5b)$$

$$\sum_{z \in \mathcal{Z}_k} w_{l,k}^z = \tau \quad \forall l \in \mathcal{L}, k \in \mathcal{K} \quad (5c)$$

$$w_{\psi,k}^z - w_{l,k}^z = 0 \quad \forall l \in \mathcal{L}, k \in \mathcal{K}, z \in \mathcal{Z}_k \quad (5d)$$

$$w_{l,k}^z \geq 0 \quad \forall l \in \mathcal{L}, k \in \mathcal{K}, z \in \mathcal{Z}_k. \quad (5e)$$

By using the constraints of the corresponding dual program, this is equivalent to the formulation

$$\max_{w_{l,k}^z} \sum_{l \in \mathcal{L}} \sum_{k \in \mathcal{K}} \sum_{z \in \mathcal{Z}_K} \hat{\mu}_{l,k}^z w_{l,k}^z \quad (6a)$$

$$\text{s.t. } (5b), (5c), (5e), \quad (6b)$$

where $\hat{\mu}_{l,k}^z$ denotes the optimal dual variables associated with the corresponding constraints (5d). This problem can now be decomposed into $|\mathcal{L}|$ leg-specific sub problems. Using $\mathcal{K}_l$ to denote the set of markets consuming resources of leg l , the exact solution of each sub problem

Table 8 Summary of results for single hub and 12 spokes instance with **R-DRA** as behavior policy

Mean μ	0.7			1.0			1.3		
Std. dev. σ	0.0	0.2	0.4	0.0	0.2	0.4	0.0	0.2	0.4
Revenue CRR w.r.t. R-DRA $\mu = 1.0$, $\sigma = 0.0$	80%	90%	92%	99%	101%	101%	102%	102%	101%
Impr. CRR over beh.	2%	36%	50%	-1%	2%	31%	-1%	0%	4%

$$V_l(c_t, t) = \max_{x_t^l} \left\{ \sum_{k \in \mathcal{K}_l} \sum_{j \in \mathcal{J}_k} q_{l,j}(x_t^l) \left(x_{t,j}^l + V_l(c_t - M_j, t+1) - V_l(c_t, t+1) \right) + V_l(c_t, t+1) \right\} \quad (7a)$$

$$V_l(0, t) = 0 \quad (7b)$$

$$V_l(c_t, 0) = 0 \quad (7c)$$

can be computed efficiently, as long as there are not too many products and prices per market, i.e., when $|\mathcal{Z}_k|$ is not too large. Note that, since (7a)–(7c) is a single-leg problem, it can be solved for each market $k \in \mathcal{K}_l$ independently. We then simply use $\hat{V}(c_t, t) = \sum_{l \in \mathcal{L}} V_l(c_t, t)$ as an approximation to the real state values when making a pricing decision according to

$$x_t^* = \operatorname{argmax}_{x_t} \left\{ \sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{J}_k} q_{l,j}(x_t) \left(x_{t,j} + \hat{V}(c_t - M_j, t+1) + \hat{V}(c_t, t+1) \right) \right\}. \quad (8)$$

6.5.4 Results

Table 8 presents the results for the modified behavior policy R-DRA in the hub and spoke instance from Sect. 6.5.2, considering $\mu \in \{0.7, 1.0, 1.3\}$ and $\sigma \in \{0, 0.2, 0.4\}$ respectively. The optimal policy cannot be computed for these instances (see Sect. 5.1.1). Results for the instances from Sect. 6.1 can be found in Appendix B.

The key findings are as follows:

- *CRR outperforms the behavior policy:* Most importantly, even in these very large instances, CRR dominates the R-DRA behavior policy, with the exception of the cases without variance in the behavior policy ($\mu = 1.0$, $\sigma = 0.0$ and $\mu = 1.3$, $\sigma = 0.0$) where CRR practically copies R-DRA and performs 1% worse. Across the tested parameterizations, the revenue improvements reached up to 50%. The trend that the improvement of CRR over R-DRA decreases with increasing μ can be explained as follows. For $\mu = 1.3$, R-DRA achieves high revenues by setting the highest admissible price very frequently, such that CRR cannot improve over R-DRA by choosing higher prices.

- *Training runtimes:* Training required 175 hours, using a single Tesla V100-SXM3-32GB GPU and 10 cores of Xeon Platinum 8168 for different environment workers. Due to the comparably small model sizes of actor and critic, the GPU utilization remained low at around 5%. These times could significantly vary depending on the chosen hyperparameters and the desired quality of the final policy, as the training process tends to decelerate towards the end. Please note that these supposedly long runtimes are no limitation in the considered application, because the training only has to be performed once. Further, once the training is done, we obtained a parameterized function which can be evaluated in real-time.

6.6 Managerial insights and limitations

Based on the described numerical results and our experiences gained from developing an effective CRR algorithm, we draw the following managerial insights:

- *Offline RL can be successfully applied to network dynamic pricing:* Most importantly, the results demonstrate that offline RL can successfully be applied to network dynamic pricing – demonstrated with a CRR algorithm. First, due to the proposed state and action space reformulation, the approach scales to larger instances with eleven legs and dozens of products. Second, as soon as there is some variation in the behavior policy, the trained CRR policy strictly dominates the respective behavior policy and realizes improvements of up to 50% in the considered instances. Thus, for practitioners, the proposed approach offers the possibility to improve their existing pricing policy, without any real-world exploration costs. The benefit of the presented approach is that it can be applied regardless of the RMS in place – solely historic sales data including the set prices is required.
- *Variation in behavior policy enables the possibility to obtain an improved policy:* The results show that CRR is only capable of finding an improved policy if the behavior policy has variation. This becomes apparent in particular through the marginal improvement of CRR when introducing variance ($\sigma = 0.1$) in the behavior policy, compared to no variance ($\sigma = 0.0$). At the same time, this marginal improvement is decreasing with increasing variance. Note however, that not any kind of variation is beneficial in general, but that variation which comprises “good” behavior is necessary. For practitioners, this means that they should consider introducing or increasing variation (moderately) for policies they expect to be suboptimal, because there is a chance that they will benefit from it when applying offline RL in the future. Especially if domain experts assume a certain bias of the behavior policy, e.g. towards too low prices, there is potential in experimenting with price variation that are not subject to this bias. Conversely, if practitioners know that there is no variation in their current policy, there is no need for applying offline RL. Note however, that offline RL could still be used to clone or parameterize a (potentially unknown) policy which is not modeled explicitly but has been applied in the past.

Despite the demonstrated and discussed high potential to improve network dynamic pricing through offline RL and in particular CRR, there remain some notable challenges.

- *Lack of a real-life case study or real-life application:* Even if the instance sizes that we consider (in the robustness analysis) are among the largest instances considered in literature, a real-life case study or actual real-life application would be insightful to assess the method's applicability.
- *Improvement bounds for CRR with strong behavior policies:* Naturally, when the behavior policy is optimal or near-optimal, there is little or no room for improvement. For example, if there is perfect knowledge with regard to the demand function structure and its parameters, the resulting behavior policy might be optimal and, thus, the CRR policy cannot outperform this benchmark. However, we would argue that perfect information is an unrealistic assumption in a real-life application.
- *Shortcomings of offline RL in general and the proposed CRR approach in specific:* Generally, offline RL is not easy to implement, because of the required difficult design choices. For the proposed CRR approach, this relates in particular to the two function approximators, the actor and the critic, because they could potentially be improved with a different model architecture, e.g. with regard to layers, nodes, etc. Moreover, the CRR algorithm requires a selection of hyperparameters, such as learning rate, batch size, filtering type, etc. The appropriate selection can be very challenging as well as time consuming and there is not guarantee that our selection is optimal.

7 Conclusion

In this work, we demonstrate that offline RL – specifically CRR – can successfully be applied to learn improved dynamic pricing policies that increase profitability, solely based on historic sales data. More specifically, we consider a network dynamic pricing problem with limited selling horizon, perishable resources, customer choice, and discrete prices. We generate the “historic” data based on state-of-the-art approaches from the literature, i.e., a DLP-based approach and, for tractable instances, based on a policy that deviates systematically from the optimal policy.

The proposed approach is built on the concept of offline RL, a comparably new concept in the realm of RL which has recently gained more attention. In particular, we demonstrate how to make an offline RL algorithm, CRR, learn improved dynamic pricing policies. The required adaptations to make this work include a reformulation of the problem's state and action spaces into low-dimensional representations. This reformulation is important beyond offline RL approaches, because it allows to apply state-of-the-art RL-based method (online and offline) to network dynamic pricing (and capacity control) problems. Due to their complexity, such network problems have not been considered for RL-based methods in the existing literature. Further, we propose an actor loss function adaptation for

the basic CRR algorithm, such that the state-dependent action space of the network dynamic pricing problem is adequately considered.

With a systematic numerical study which uses instances from the key literature on pricing and revenue management, we demonstrate that the offline RL algorithm outperforms the behavior policy, with which the sales data was generated, by up to 50%. Notably, the offline RL algorithm requires no information on customer arrival probabilities and customer choice model parameters – in contrast to the benchmarks. It is shown that the proposed approach scales to larger instances with eleven resources and 36 products. The results further show that higher variation in the behavior policy increases the chances that offline RL learns a better policy, given that the variation includes pricing decisions that resulted in high revenues. This suggests that practitioners considering to apply the proposed concept should assess first whether there is variability in their current pricing and to which extend increasing the variation for future benefits based on offline RL policies is reasonable.

Despite these limitations, we believe that, overall, offline RL can play a central role in the future of pricing and revenue management, in particular in dynamic pricing and capacity control – both in academia and practice. The beauty of the offline RL concept lies in the fact that it opens the door for applying powerful learning-based algorithms also in pricing and revenue management applications where making errors is unacceptable. It solves the two central issues that currently prevent using the existing powerful online RL algorithms in these applications. That is, on the one hand, directly training in the real world is too costly – due to lost revenues – and, on the other hand, modeling the real world in a simulation inevitably comes with model misspecifications which limit the trained policy's performance in the eventual application. Overall, offline RL is a cost-free approach to improve the pricing and, thus, to increase profitability.

For future work, we see the following three important paths. First, it would be insightful for practitioners to quantify the lost revenues when actually applying online RL, like Soft Actor-Critic or Proximal Policy Optimization. Second, an extension of our approach – which presumably is promising to further improve the results – is to fine-tune the CRR policy online. The general idea is to combine the benefits of offline RL and online RL by determining an effective policy first through the cost-free offline RL approach and to improve the policy further by applying online RL in the actual environment. Third, the mentioned topic of offline evaluation is a reasonable next step in the context of offline RL-based pricing or capacity control.

Appendix A: Modified Instance

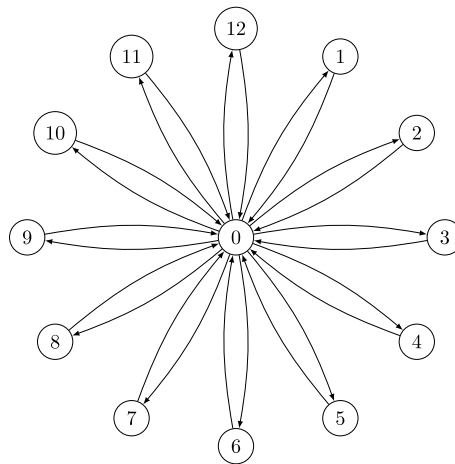


Fig. 16 Network structure of the modified instance. The network takes the form of a single hub and $N = 12$ spokes network

Appendix B: Results modified behavior policy in large instance

Table 9 presents the results for the modified behavior policy R-DRA (Sect. 6.5.3) in the large instance from Sect. 6.1.

It is plausible that the deterministic, unbiased DRA policy, which operates under perfect knowledge, i.e., with the actual demand function, is close to the optimal policy. This is evident as CRR approaches 100% revenue realization across various parameterizations, but it does not exceed this threshold.

Table 9 Summary of results for large instance with **R-DRA** as behavior policy

Mean μ	0.7			1.0			1.3		
Std. dev. σ	0.0	0.2	0.4	0.0	0.2	0.4	0.0	0.2	0.4
Revenue CRR w.r.t. R-DRA $\mu = 1.0$, $\sigma = 0.0$	85%	98%	98%	99%	99%	99%	95%	97%	99%
Impr. CRR over beh.	0%	27%	35%	-1%	9%	21%	1%	7%	19%

Appendix C: Overview instances in the literature

Table 10 summarizes the relevant literature on network *pricing* and revenue management (*capacity control/offer set* optimization) (for details on the various optimization problems in the context of revenue management, see e.g. Klein et al. (2020b)). With regard to the network structure and size, we report the largest considered instance for each work. Regarding the structure, one can differ between *hub-and-spoke* structures (i.e., one hub

Table 10 Comparison of considered instance sizes in literature on network dynamic pricing and revenue management

Paper	Considered problem	Largest instance		
		Network structure	Number of resources/ legs	Number of prod- ucts
Besbes and Zeevi (2012)	Pricing	Custom	3	2
Ferreira et al. (2018)	Pricing	Custom	3	2
Jasin and Kumar (2012)	Offer set	Custom	4	6
Barz et al. (2023)	Pricing	Custom	5	11
An et al. (2021)	Capacity control	Custom	11	15
Liu and Van Ryzin (2008)	Offer set	Custom hub-and-spoke	7	22
Bront et al. (2009)	Offer set	Custom hub-and-spoke	7	22
Meissner and Strauss (2012)	Offer set	Custom hub-and-spoke	7	22
this work (Sections 6.1-6.4)	Pricing	Custom	11	36
Zhang and Lu (2013)	Pricing	Single hub, N spokes	16	80
Vossen and Zhang (2015)	Offer set	Single hub, N spokes	16	144
Topaloglu (2009)	Offer set	Single hub, N spokes	16	144
Erdelyi and Topaloglu (2011)	Offer set	Single hub, N spokes	16	144
Zhang (2011)	Offer set	Single hub, N spokes	24	168
Bertsimas and De Boer (2005)	Capacity control	Single hub, N spokes	30	240
Zhang and Adelman (2009)	Offer set	Single hub, N spokes	32	272
this work (Section 6.5)	Pricing	Single hub, N spokes	24	336
Adelman (2007)	Capacity control	Single hub, N spokes	40	420
Perakis and Roels (2010)	Capacity control	Not fully disclosed	“517 itineraries”	
Van Ryzin and Vulcano (2008)	Capacity control	Not fully disclosed	“62 legs, 1844 origin-destination-fare-classes”	
Birbil et al. (2014)	Capacity control	Not fully disclosed	“5000 OD-pairs”	

and multiple spokes) and self-designed *custom* structures (multiple hubs and spokes). Not all of the works *fully disclose* the specific network structure. With regard to the size, we report the *number of legs* and the (resulting) *number of products*. The papers in Table 12 are ordered according to the number of products.

As the table shows, the largest instance in the new setting that we consider as part of the robustness analysis is among the largest considered in literature which are fully disclosed. More specifically, we consider a hub-and-spoke network structure with one hub and 12 spokes, resulting in 336 products (2 products per leg). The only work with more legs and products is Adelman (2007) where, in contrast to our work and in contrast to its successor work (Zhang and Adelman 2009), there is no customer-choice behavior considered endogenously in the optimization.

Funding Open Access funding enabled and organized by Projekt DEAL.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Adelman D (2007) Dynamic bid prices in revenue management. *Op Res* 55(4):647–661
- An J, Mikhaylov A, Jung S-U (2021) A linear programming approach for robust network revenue management in the airline industry. *J Air Transp Manag* 91:101979
- Barz C, Laumer S, Freyschmidt M, Martínez-Blanco J (2023) Discrete dynamic pricing and application of network revenue management for flibus. *J Rev Pric Manag* 22(1):16–33
- Bertsimas D, De Boer S (2005) Simulation-based booking limits for airline revenue management. *Op Res* 53(1):90–106
- Besbes O, Zeevi A (2012) Blind network revenue management. *Op Res* 60(6):1537–1550
- Birbil Şİ, Frenk J, Gromicho JA, Zhang S (2014) A network airline revenue management framework based on decomposition by origins and destinations. *Transp Sci* 48(3):313–333
- Bitran G, Mondschein S (1997) Periodic pricing of seasonal products in retailing. *Manag Sci* 43(1):64–79
- Bondoux N, Quan A, Fiig T, Acuna-Agost R (2020) Reinforcement learning applied to airline revenue management. *J Rev Pric Manag* 19(5):332–348
- Bront JJM, Méndez-Díaz I, Vulcano G (2009) A column generation algorithm for choice-based network revenue management. *Op Res* 57(3):769–784
- Bu J, Simchi-Levi D, Xu Y (2020) Online pricing with offline data: Phase transition and inverse square law. In international conference on machine learning, pp 1202–1210. PMLR
- den Boer AV (2015) Dynamic pricing and learning: historical origins, current research, and new directions. *Surv Op Res Manag Sci* 20(1):1–18
- Dror M, Trudeau P, Ladany SP (1988) Network models for seat allocation on flights. *Transp Res Part B: Methodol* 22(4):239–250
- Elmaghraby W, Keskinocak P (2003) Dynamic pricing in the presence of inventory considerations: research overview, current practices, and future directions. *Manag Sci* 49(10):1287–1309
- Erdelyi A, Topaloglu H (2011) Using decomposition methods to solve pricing problems in network revenue management. *J Rev Pric Manag* 10:325–343

- Ferreira KJ, Simchi-Levi D, Wang H (2018) Online network revenue management using thompson sampling. *Op Res* 66(6):1586–1602
- Gallego G, Topaloglu H (2019) Revenue management and pricing analytics. Springer, New York, NY
- Gallego G, van Ryzin GJ (1997) A multiproduct dynamic pricing problem and its applications to network yield management. *Op Res* 45(1):24–41
- Gallego G, Iyengar G, Phillips R, Dubey A (2004) Managing flexible products on a network. Technical report, CORC Technical Report Tr-2004-01
- Glover F, Glover R, Lorenzo J, McMillan C (1982) The passenger-mix problem in the scheduled airlines. *Interfaces* 12(3):73–80
- Gosavi A (2004) A reinforcement learning algorithm based on policy iteration for average reward: empirical results with yield management and convergence analysis. *Mach Learn* 55:5–29
- Gosavi A, Bandla N, Das TK (2002) A reinforcement learning approach to a single leg airline revenue management problem with multiple fare classes and overbooking. *IIE Trans* 34:729–742
- Greg B, Vicki C, Ludwig P, Jonas S, John S, Jie T, Wojciech Z (2016) Openai gym. <https://gym.openai.com>
- Haarnoja T, Zhou A, Hartikainen K, Tucker G, Ha S, Tan J, Kumar V, Zhu H, Gupta A, Abbeel P, Levine S (2019) Soft actor-critic algorithms and applications
- Jasin S, Kumar S (2012) A re-solving heuristic with bounded revenue loss for network revenue management with customer choice. *Math Op Res* 37(2):313–345
- Kastius A, Schlosser R (2022) Dynamic pricing under competition using reinforcement learning. *J Rev Pric Manag* 21(1):50–63
- Klein R, Koch S, Steinhardt C, Strauss AK (2020) A review of revenue management: recent generalizations and advances in industry applications. *Eur J Op Res* 284(2):397–412
- Klein R, Koch S, Steinhardt C, Strauss AK (2020) A review of revenue management: recent generalizations and advances in industry applications. *Eur J Op Res* 284(2):397–412
- Kumar A, Fu J, Tucker G, Levine S (2019) Stabilizing off-policy q-learning via bootstrapping error reduction. In *Advances in Neural Information Processing Systems*
- Kumar A, Zhou A, Tucker G, Levine S (2020) Conservative q-learning for offline reinforcement learning. *Adv Neural Inf Process Syst* 33:1179–1191
- Lawhead RJ, Gosavi A (2019) A bounded actor-critic reinforcement learning algorithm applied to airline revenue management. *Eng Appl Artif Intell* 82:252–262
- Levine S, Finn C, Darrell T, Abbeel P (2016) End-to-end training of deep visuomotor policies. *J Mach Learn Res* 17:1–40
- Levine S, Kumar A, Tucker G, Fu J (2020) Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arxiv*
- Liang E, Liaw R, Nishihara R, Moritz P, Fox R, Goldberg K, Gonzalez JE, Jordan MI, Stoica I (2018) Abstractions for distributed reinforcement learning. In *International Conference on Machine Learning (ICML)*
- Liu Q, Van Ryzin G (2008) On the choice-based linear programming model for network revenue management. *Manuf Serv Op Manag* 10(2):288–310
- Meissner J, Strauss A (2012) Network revenue management with inventory-sensitive bid prices and customer choice. *Eur J Op Res* 216(2):459–468
- Misra K, Schwartz EM, Abernethy J (2019) Dynamic online pricing with incomplete information using multiarmed bandit experiments. *Mark Sci* 38(2):226–252
- Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G et al (2015) Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533
- Mnih V, Badia AP, Mirza L, Graves A, Harley T, Lillicrap TP, Silver D, Kavukcuoglu K (2016) Asynchronous methods for deep reinforcement learning. In *33rd international conference on machine learning*, pp 2850–2869. PMLR
- Monier L, Kmec J, Laterre A, Pierrot T, Courgeau V, Sigaud O, Beguir K (2020) Offline reinforcement learning hands-on. *arxiv*
- Özer Ö, Phillips R (2012) The Oxford handbook of pricing management. Oxford University Press, Oxford, United Kingdom
- Perakis G, Roels G (2010) Robust controls for network revenue management. *Manuf Serv Op Manag* 12(1):56–76
- Powell W (2016) Perspectives of approximate dynamic programming. *Ann Op Res* 241:319–356
- Precup D, Sutton RS, Singh S (2000) Eligibility traces for off-policy policy evaluation. *Computer Sci Dep Fac Publ Series*. 2000:759–766

- Prudencio RF, Maximo MROA, Colombini EL (2023) A survey on offline reinforcement learning: taxonomy, review, and open problems. *IEEE Trans Neural Netw Learn Syst* 35:10237–10257
- Pumpensanti P, Wang H (2020) A re-solving heuristic with uniformly bounded loss for network revenue management. *Manag Sci* 66(7):2801–3294
- Raju CVL, Narahari Y, Ravikumar K (2006) Learning dynamic prices in electronic retail markets with customer segmentation. *Ann Op Res* 143:59–75
- Rana R, Oliveira FS (2014) Real-time dynamic pricing in a non-stationary environment using model-free reinforcement learning. *Omega* 47:116–126
- Rummery GA, Niranjan M (1994) On-line q-learning using connectionist systems. Technical report, University of Cambridge, Department of Engineering, Cambridge, England
- Schön C (2010) Optimal dynamic price selection under attraction choice models. *Eur J Op Res* 205(3):650–660
- Silver D, Huang A, Maddison CJ, Guez A, Sifre L, van den Driessche G, Schrittwieser J, Antonoglou I, Panneershelvam V, Lanctot M et al (2016) Mastering the game of go with deep neural networks and tree search. *Nature* 529(7587):484–489
- Strauss AK, Klein R, Steinhardt C (2018) A review of choice-based revenue management: theory and methods. *Eur J Op Res* 271(2):375–387
- Sutton RS, Barto AG (2018) *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA
- Talluri KT, van Ryzin GJ (2004) *The theory and practice of revenue management*. Springer, Boston, MA
- Talluri KT, van Ryzin GJ (2004) Revenue management under a general discrete choice model of consumer behavior. *Manag Sci* 50(1):15–33
- Thomas P, Theodorou G, Ghavamzadeh M (2015) High-confidence off-policy evaluation. In *Twenty-Ninth AAAI conference on artificial intelligence*
- Topaloglu H (2009) Using lagrangian relaxation to compute capacity-dependent bid prices in network revenue management. *Op Res* 57(3):637–649
- Van Ryzin G, Vulcano G (2008) Simulation-based optimization of virtual nesting controls for network revenue management. *Op Res* 56(4):865–880
- Vossen TW, Zhang D (2015) Reductions of approximate linear programs for network revenue management. *Op Res* 63(6):1352–1371
- Wang Z, Novikov A, Zolna K, Merel JS, Springenberg JT, Reed SE, Shahriari B, Siegel N, Merel J, Gulcehre C et al (2020) Critic regularized regression. *Neural Inf Process Syst* 33:7768–7778
- Williams RJ (1992) Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach Learn* 8:229–256
- Zhang D (2011) An improved dynamic programming decomposition approach for network revenue management. *Manuf Serv Op Manag* 13(1):35–52
- Zhang D, Adelman D (2009) An approximate dynamic programming approach to network revenue management with customer choice. *Transp Sci* 43(3):381–394
- Zhang D, Lu Z (2013) Assessing the value of dynamic pricing in network revenue management. *INFORMS J Computing* 25(1):102–115

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Authors and Affiliations

Philipp Hausenblas¹  · Dominik Eichhorn¹  · Andreas Brieden¹  ·
Matthias Soppert²  · Claudius Steinhardt² 

✉ Matthias Soppert
matthias.soppert@unibw.de

Philipp Hausenblas
philipp.hausenblas@unibw.de

Dominik Eichhorn
dominik.eichhorn@unibw.de

Andreas Brieden
andreas.brieden@unibw.de

Claudius Steinhardt
claudius.steinhardt@unibw.de

- ¹ Chair of Data Analytics & Statistics, University of the Bundeswehr Munich, Neubiberg, Germany
- ² Chair of Business Analytics & Management Science, University of the Bundeswehr Munich, Neubiberg, Germany

Terms and Conditions

Springer Nature journal content, brought to you courtesy of Springer Nature Customer Service Center GmbH ("Springer Nature").

Springer Nature supports a reasonable amount of sharing of research papers by authors, subscribers and authorised users ("Users"), for small-scale personal, non-commercial use provided that all copyright, trade and service marks and other proprietary notices are maintained. By accessing, sharing, receiving or otherwise using the Springer Nature journal content you agree to these terms of use ("Terms"). For these purposes, Springer Nature considers academic use (by researchers and students) to be non-commercial.

These Terms are supplementary and will apply in addition to any applicable website terms and conditions, a relevant site licence or a personal subscription. These Terms will prevail over any conflict or ambiguity with regards to the relevant terms, a site licence or a personal subscription (to the extent of the conflict or ambiguity only). For Creative Commons-licensed articles, the terms of the Creative Commons license used will apply.

We collect and use personal data to provide access to the Springer Nature journal content. We may also use these personal data internally within ResearchGate and Springer Nature and as agreed share it, in an anonymised way, for purposes of tracking, analysis and reporting. We will not otherwise disclose your personal data outside the ResearchGate or the Springer Nature group of companies unless we have your permission as detailed in the Privacy Policy.

While Users may use the Springer Nature journal content for small scale, personal non-commercial use, it is important to note that Users may not:

1. use such content for the purpose of providing other users with access on a regular or large scale basis or as a means to circumvent access control;
2. use such content where to do so would be considered a criminal or statutory offence in any jurisdiction, or gives rise to civil liability, or is otherwise unlawful;
3. falsely or misleadingly imply or suggest endorsement, approval, sponsorship, or association unless explicitly agreed to by Springer Nature in writing;
4. use bots or other automated methods to access the content or redirect messages
5. override any security feature or exclusionary protocol; or
6. share the content in order to create substitute for Springer Nature products or services or a systematic database of Springer Nature journal content.

In line with the restriction against commercial use, Springer Nature does not permit the creation of a product or service that creates revenue, royalties, rent or income from our content or its inclusion as part of a paid for service or for other commercial gain. Springer Nature journal content cannot be used for inter-library loans and librarians may not upload Springer Nature journal content on a large scale into their, or any other, institutional repository.

These terms of use are reviewed regularly and may be amended at any time. Springer Nature is not obligated to publish any information or content on this website and may remove it or features or functionality at our sole discretion, at any time with or without notice. Springer Nature may revoke this licence to you at any time and remove access to any copies of the Springer Nature journal content which have been saved.

To the fullest extent permitted by law, Springer Nature makes no warranties, representations or guarantees to Users, either express or implied with respect to the Springer nature journal content and all parties disclaim and waive any implied warranties or warranties imposed by law, including merchantability or fitness for any particular purpose.

Please note that these rights do not automatically extend to content, data or other material published by Springer Nature that may be licensed from third parties.

If you would like to use or distribute our Springer Nature journal content to a wider audience or on a regular basis or in any other manner not expressly permitted by these Terms, please contact Springer Nature at

onlineservice@springernature.com